Efficient On-Device Domain Learning for Keyword Spotting on Ultra-Low-Power Platforms

Cristian Cioflan, *Graduate Student Member, IEEE*, Lukas Cavigelli, *Member, IEEE*, Manuele Rusci, Miguel de Prado, and Luca Benini, *Fellow, IEEE*

***Abstract*—Deep Neural Network (DNN)-based keyword spotting (KWS) accuracy degrades in noisy environments. On-site adaptation to previously unseen noise is crucial to recover accuracy loss, and On-Device Learning (ODL) is required in scenarios where adaptation has to happen in the field. In this work, we propose a fully On-Device Domain Adaptation (ODDA) system, enabling edge devices to achieve noise-robust KWS. We achieve up to 14% accuracy gains over already-robust KWS models and up to 21% increments when evaluating our methodology on keyword datasets disjoint from the offline training set. In extreme edge scenarios where KWS is critical, using as little as 10 kB of memory and only 100 labeled utterances, we enable ODL and demonstrate accuracy recovery of up to 5% after adapting to complex, non-stationary speech noise. We show that domain adaptation can be achieved on ultra-low-power (ULP) microcontrollers with as little as 357 mJ within 14 s on always-on, battery-operated devices. This work is the first to demonstrate an end-to-end ODDA system for noise-robust KWS models on ULP, extreme edge platforms.**

***Index Terms*—Domain adaptation, extreme edge, keyword spotting (KWS), low-power microcontrollers, noise robustness, on-device learning (ODL), TinyML.**

I. Introduction

KEYWORD spotting [1] entails processing an acoustic input to recognize a target word (i.e., keyword) from a predefined set, a task also known as closed-vocabulary speech recognition. The keyword spotting (KWS) task nowadays is mostly carried out by leveraging Deep Neural Networks (DNNs) [2], [3], [4], [5], [6], [7], [8], [9], [10] and is extremely critical in a plethora of extreme edge deployment scenarios to control voice assistants [11], [12], [13] or for human–robot interaction [14], [15], [16]. While compact DNN models tailored for edge deployment on ultra-low-power (ULP) hardware achieve high classification accuracy on curated datasets within clean, noiseless environments [10], [11], [17], [18], their accuracy decreases significantly (i.e., up to 27% [19]) when the data used to train the model offline differ from the inputs received in the deployment scenario. The accuracy losses can arise due to the environment in which the KWS system is used, for example, linear distortions produced by additive background noises [19] or convolutional distortions caused by the room reverberation profile [5] or by the microphone's characteristics [20]. Additionally, the system's accuracy can be impacted by the speakers' gender [21] or accent [22].

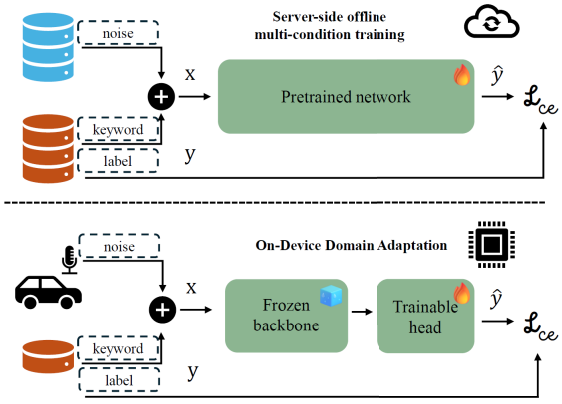


Fig. 1. ODDA overview. During the server-side stage, the KWS model is pretrained with noise-augmented samples. The domain adaptation takes place on the device, where only the head is trained. For training, a subset of the KWS dataset is used, augmented with noise (e.g., traffic) acquired on-site.

Works addressing the robustness to noise of edge DNN-based KWS algorithms are often based on training techniques, such as artificially augmenting clean utterances with noise during the training stage [6], [7], [8], [23], [24]. Alternatively, Jung et al. [3], Huang et al. [25], [26], and Rusci et al. [27] proposed to learn the noise profile to filter it away from the input signal. In both scenarios, the models are designed and trained offline, on server machines, to achieve noise robustness, while meeting the resource constraints of the target device. The constraints of extreme edge platforms are: the storage memory of few MB for retaining the parameters of the DNN, the memory required for computing the activations

Received 11 November 2025; revised 15 December 2025; accepted 31 December 2025. Date of publication 14 January 2026; date of current version 9 March 2026. This work supported in part by the Swiss National Science Foundation under Grant 207913: TinyTrainer: On-chip Training for TinyML devices; in part by the Swiss State Secretariat for Education, Research, and Innovation (SERI) under the SwissChips initiative; and in part by the Horizon Europe dAIEdge Grant 101120726. *(Corresponding author: Cristian Cioflan.)*

Cristian Cioflan is with the Integrated Systems Laboratory, Department of Information Technology and Electrical Engineering (D-ITET), ETH Zürich, 8092 Zürich, Switzerland (e-mail: cioflanc@iis.ee.ethz.ch; lbenini@iis.ee.ethz.ch).

Lukas Cavigelli is with the Computing Systems Laboratory, Zurich Research Center, Huawei Technologies, 8050 Zürich, Switzerland (e-mail: lukas.cavigelli@huawei.com).

Manuele Rusci is with the Department of Electrical Engineering (ESAT), KU Leuven, 3000 Leuven, Belgium (e-mail: manuele.rusci@kuleuven.be).

Miguel de Prado is with VERSES, Los Angeles, CA 90016 USA (e-mail: miguel.deprado@verses.ai).

Luca Benini is with the Integrated Systems Laboratory, Department of Information Technology and Electrical Engineering (D-ITET), ETH Zürich, 8092 Zürich, Switzerland, and also with the Department of Electrical, Electronic, and Information Engineering (DEI), University of Bologna, 40126 Bologna, Italy.

Digital Object Identifier 10.1109/JIOT.2026.3654437

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

of each layer (i.e., hundreds of kB to few MB), the latency associated with real-time inference (i.e., tens of ms), as well as the energy consumption for always-on, battery-operated devices with tens of mW of power [28], [29]. Therefore, the works analyzing the deployment of DNNs on resource-constrained embedded systems—a theme deeply investigated by the recently born TinyML community—only focus on optimizing the inference task [30], [31], [32].

In contrast, this work focuses on enabling On-Device Learning (ODL) in a TinyML scenario, that is, performing DNN training on extreme edge platforms, namely microcontroller unit (MCU) systems-on-chip with power consumption of a few tens of mW and on-device memory of a few MB. We enable these low-cost, low-power Internet of Things (IoT) devices [33] to adapt to new environmental noises that affect KWS systems under on-site domain distribution shifts. When relying on Gradient Descent (GD) through backpropagation, the limited memory available on MCUs must additionally store gradient vectors for the update of the weight parameters, increasing the memory cost compared to the inference task. A similar challenge concerns the computational cost of the DNN training steps, which directly translates into offline time and reduced battery life. It is hence clear that the complexity of the models and learning algorithms should be reduced, while additionally ensuring fast convergence with a low number of labeled input samples.

A few frameworks have been proposed [34], [35], [36], [37], [38], introducing memory- and latency-oriented optimizations to enable ODL in TinyML scenarios, and thus updating all or a subset of the DNN parameters on embedded platforms. Such preliminary approaches contribute toward achieving efficient post-deployment adaptation on low-power and ULP platforms. At the same time, noise-aware pretrained KWS models have been implemented on off-the-shelf MCUs [39] as well as on Application-Specific Integrated Circuits (ASICs) [40], [41], including neuromorphic chips [42], leveraging their extremely high efficiency [43], [44]. Nevertheless, ULP on-board update for KWS in the context of on-site, real-world data distribution shifts, and particularly environmental audio noises, remains an open problem, which has not been addressed by these previous works.

We introduce On-Device Domain Adaptation (ODDA), depicted in Fig. 1, an ODL methodology for adapting a KWS model already deployed on a ULP device to novel noise domains. We consider KWS datasets, which we augment with on-site, real-world recordings, thus producing noisy utterances employed to fine-tune a model deployed on the target platform. We demonstrate our approach on memory-constrained extreme edge devices, allowing for flexible adaptation depths in the presence of speech noise.

This work extends our early work on ODDA [2] by providing a complete description and analysis of our ODDA methodology and its embodiment on ULP MCUs. Furthermore, all contributions regarding the evaluation of ODDA on coarticulated keywords, topological and learning strategy optimizations, heterogeneous deployment and accelerator integration, and evaluation of resource utilization and power consumption are original to this work. In detail, our contributions are as follows.

1) We propose an ODDA methodology producing noise-robust KWS systems.
2) We demonstrate the practical benefits of the proposed methodology on resource-constrained, TinyML systems by evaluating it on a ULP hardware platform [i.e., the GAP9 System-on-Chip (SoC)[1]].
3) We propose a mixed-precision, heterogeneous deployment strategy optimizing the ODL pipeline, by efficiently offloading the integerized backbone operations to a convolutional accelerator and distributing the trainable classifier operations to a multicore cluster.
4) We extensively evaluate our methodology on synthetically augmented noisy versions of public datasets containing manually recorded keywords and extracted coarticulated keywords.

This work is the first to demonstrate a complete ODDA system for KWS models in ULP (i.e., tens of mW) systems. We measure accuracy gains up to 14% in previously unseen noisy environments compared to an offline-trained, noise-robust model. We furthermore evaluate our methodology on datasets disjoint from the offline training dataset, with accuracy increments of up to 21%. Our memory- and storage-reduction strategies make ODDA suitable for data-scarce conditions (i.e., only ten labeled utterances per class); with read–write memory requirements as low as 10 kB, we improve by up to 5% the accuracy in challenging *meeting* noise conditions. We demonstrate, in practice, the constrained ODDA on an ULP, multicore GAP9 MCU, enabling on-site domain adaptation at the modest cost of 178 μJ per sample for an average power consumption of 26 mW. Considering an always-on, battery-operated device, and a 3.7 V, 100 mAh battery, a system implementing our proposed ODDA methodology could learn a new noise in only 13.5 s and then offer robust predictions for up to 27.5 h.

The rest of this article is organized as follows. In Section II, previous works on extreme edge KWS and noise-robust KWS are introduced and discussed, followed by ODL frameworks and algorithms and ULP platforms suitable for KWS. Section III introduces our domain adaptation methodology and presents the system and the implementation details of the proposed methodology. In Section IV we evaluate our ODDA on noise-augmented KWS datasets. Section V presents and discusses the hardware-associated costs of our ODL methodology. Finally, Section VII concludes this article, summarizing the contributions and results.

II. Related Work

In this section, we review the literature on KWS topologies and noise-robust KWS systems. We then give an overview of the state-of-the-art ODL frameworks targeting extreme edge, battery-operated devices.

A. KWS Topologies

Near-sensor KWS applications call for DNN topologies targeting extreme edge scenarios. Early works by [45], [46], [47], and [48] compared recurrent, convolutional, and hybrid

[1]https://greenwaves-technologies.com/gap9_processor/

TABLE I

SELECTION OF THE MOST SIGNIFICANT WORKS ADDRESSING NOISE-ROBUST KWS IN ENVIRONMENTS WITH SNR ≤ 0 DB. THE DATASETS SHOWN ARE THE KWS DATASET, FOLLOWED BY THE NOISE DATASET USED FOR AUGMENTATION; N.P.A. STANDS FOR "NOT PUBLICLY AVAILABLE." WORKS THAT DO NOT DISCLOSE THE COMPUTATIONAL COMPLEXITY OR THE NUMBER OF PARAMETERS ARE TREATED AS "N.A." (I.E., "NOT AVAILABLE") ONE MAC ACCOUNTS FOR TWO FLOPS. WHEN THE RESULTS ARE REPRESENTED THROUGH THE RECEIVER OPERATING CHARACTERISTIC (ROC) CURVES [E.G., FRR OR EQUAL ERROR RATE (EER)], WE REPORT THE MOST FAVORABLE VALUES. FOR EACH WORK, WE REPORT THE PERFORMANCE IMPROVEMENT OVER THEIR SELECTED BASELINE

Reference	Method	# keywords	Dataset	Comp. complexity	Parameters	Metric	Gain over baseline
Ji *et al.* 2020 [65]	BF†	1	N.P.A. / N.P.A.	N.A.	940 k	FRR	0.5
Ji *et al.* 2021 [66]	BF†	1	N.P.A. / N.P.A.	N.A.	3.3 M	FRR	0.1
Huang *et al.* 2019 [26]	ANC†	2	N.P.A. / N.P.A.	16 MMAC	160 k	FRR	0.15
Jung *et al.* 2020 [3]	SE†	35	GSC / [58]	N.A.	N.A.	EER	0.46%
Ng *et al.* 2023 [9]	SE†	1	[59] / [59]	N.A.	622 k	Acc.	8.6%
Yang *et al.* 2023 [5]	SE†	12	GSC / [58]	N.A.	50.5 k	Acc.	12.93%
Casebeer *et al.* 2024 [4]	SE†	35	GSC / [60]	200 MFLOP	300 k	F1	0.17
Chen *et al.* 2025 [67]	SE†	12	GSC / [58]	13.4 MMAC	17.6 k	Acc.	8.8%
Mohanty *et al.* 2021 [23]	MCT‡	12	GSC / N.P.A.	7.5 MMAC	34.5 k	Acc.	10%
López-Espejo *et al.* 2021 [6]	MCT‡	11	GSC / [61]	894 MMAC	238 k	Acc.	2.47%
Park *et al.* 2021 [24]	MCT‡	1	N.P.A. / N.P.A.	N.A.	N.A.	FRR	2.2%
Ng *et al.* 2022 [7]	MCT‡	12	GSC / [58]	22.2 MMAC	119 k	Acc.	4.94%
Ng *et al.* 2022 [8]	MCT‡	10	GSC / [62]	N.A.	607 k	Acc.	1.5%
Xi *et al.* 2024 [52]	MCT‡	20	[56] / [63]	N.A.	2 M	Recall	0.1
Dai *et al.* 2025 [68]	MCT‡	12	GSC / N.P.A.	N.A.	N.A.	Acc.	3.6%
Lim *et al.* 2023 [69]	MCT&KD‡	12	GSC / [58]	8.5 MMAC	27.3 k	Acc.	5%
Mork *et al.* 2024 [70]	MCT&KD‡	35	GSC / [64]	53.8 MFLOP	607 k	Acc.	15.6%
Ours	**ODDA‡**	**12**	**GSC / [57]**	2.95 MFLOP	23 k	**Acc.**	**12.84%**

† Front-end method. ‡ Back-end method.

implementations of neural networks. These works provided a baseline on the Google Speech Commands (GSC) dataset, containing KWS samples acquired in quiet environments.

Instead of relying on hand-crafted, traditional topologies, [49] and [50] employ Neural Architecture Search (NAS) algorithms to build the DNN using a convolutional and linear search space. Recently, attention-based networks [51], transducers [52], and Transformer architectures [17], [18], [53] were employed in KWS, as their context awareness is beneficial for time-series classification. By replacing the traditional attention mechanism with linear attention, Scherer et al. [10] achieved state-of-the-art accuracy on the GSC dataset while ensuring the feasibility of extreme edge inference.

Although such works achieve strong classification performance on clean, noiseless datasets, they lack the robustness to perform reliably in noisy environments. Moreover, only a few of them demonstrated an efficient accuracy-complexity trade-off, suited for on-device implementation [2], [10], [39], [54]. To address this gap, we propose a methodology capable of robustifying KWS models on extreme edge devices, and we demonstrate it for the Depthwise Separable Convolutional Neural Network (DS-CNN) topology, which has been conceived for ULP extreme edge platforms [2].

B. Noise-Robust KWS

In analyzing previous approaches for noise-robustKWS, we follow the taxonomy introduced by López-Espejo et al. [1], primarily analyzing works explicitly addressing background additive noise, but also considering the combined effect of said noise in far-field conditions. Concretely, we will start with *front-end methods*, which preprocess and denoise a noise-submerged speech signal before feeding it to a generic DNN, followed by Multi-Condition Training (MCT) *back-end methods*, which propose acoustic models with enhanced generalization capabilities in diverse adverse acoustic conditions. The works, which use either public (i.e., GSC [22], [55], [56]) or internal keyword datasets, as well as noise datasets (i.e., Diverse Environments Multichannel Acoustic Noise Database (DEMAND) [57], MUSAN [58], and others [59], [60], [61], [62], [63], [64]), are summarized in Table I.

1) Front-End Methods: Beamforming (BF) (i.e., emphasizing a signal received from the target direction and suppressing interference sources from other directions using microphone arrays) was proposed in systems containing microphone arrays. Ji et al. [65] and Yu et al. [71] proposed multichannel BF systems following multiple targets and, similar to [66], they jointly trained the BF component and the KWS model. While suitable for BF, [9] demonstrates that Speech Enhancement (SE) front ends alone can improve the noise robustness of microphone-array-based KWS systems, using multichannel ConvMixer blocks.

Works such as [25] and [26] require two microphones to perform Active Noise Cancellation (ANC). They aim to address the cocktail party problem (i.e., the ability to focus on an individual conversation, while filtering out other discussions and surrounding noises) by presenting the noise profile to an adaptive filter, which learns to subtract the noise patterns (e.g, music and TV noise) from the corrupted input signal, thus effectively denoising it.

Although efficiently addressing background noise and demonstrating False Rejection Rate (FRR) reductions of up to 0.5, the main limitation of such methods is the requirement of multiple microphones to exploit spatial information. In contrast to these works, our methodology enables KWS systems to gain noise robustness on single-microphone devices, thus reducing the device's size and cost. Furthermore, while the attainable accuracy of previous works is limited by the type of noises seen at training time, we improve our models' performance (i.e., by 14% in *speech* noise conditions for a Signal-to-Noise Ratio (SNR) of 0 dB) by specializing on the target noise.

SE methods aim at estimating and subtracting the noise component from the input signal before reaching the acoustic model. Jung et al. [3] and Rusci et al. [27] proposed multitask networks trained jointly, in which KWS acoustic models are preceded by learned filters. Yang et al. [5] additionally computed the speech presence probability to apply KWS on interest signals, whereas Casebeer et al. [4] replaced the trained noise filter with a meta-adaptive filter trained with a learning-to-learn strategy.

Although these works eliminate the need for microphone arrays, they increase the latency and dimension of the KWS system, given that the acoustic model has to be preceded by the front-end module. To address this issue, Chen et al. [67] proposed a parallel network where the SE component enriches the latent representations of the KWS branch during training time, whereas the inference relies only on the KWS branch. Nonetheless, neither [67] nor the other SE works support post-deployment model fine-tuning, as the acoustic models' parameters are fixed at deployment time. In contrast with these works, our system consists of a single acoustic model, end-to-end adaptable on edge platforms.

2) Back-End Methods: Significant effort has been spent in the direction of robustifying KWS networks against noise, thus mitigating the front-end cost while enhancing the classification performance in adverse conditions. MCT consists of training the acoustic model with clean utterances augmented with the distortions estimated to be present in the test environment. In the case of noise-robust KWS, the speech signal is overlapped with additive background noise, additionally modeling the noise levels in the environment through the SNR [6], [23]. Recent works replace traditional training techniques with efficient learning strategies (e.g., curriculum learning [7], [69], contrastive learning [8], [68], and triplet loss-based learning [72]) to improve the model robustness to adverse conditions, at the cost of more complex data management [73], [74]. Furthermore, alternative strategies to supervised learning techniques have been proposed, such as self-supervised learning [70] or unsupervised learning [68]. They additionally expose the models to multiple SNR levels [7] or different languages [72] during the training process.

Drawing inspiration from the work of Kim et al. [75] on prototypical Knowledge Distillation (KD) (i.e., transferring the knowledge of a large teacher network to a smaller student model), Lim and Baek [69] proposed a KD strategy aimed at producing compressed noise-robust KWS models. While the student network has only 27.3k parameters, the curriculum-based MCT teacher has 321k parameters, and the distillation process is performed offline.

To improve the latent representation of the acoustic model, Mork et al. [70] employed a self-supervised pretraining pipeline during the student–teacher KD, with a student capacity of 607k parameters. On the other hand, while noise samples are employed in KWS meta-learning [76], [77], existing works do not address noise robustness.

In Table I, we show MCT works relying on complex learning strategies such as curriculum learning [7], [69] or contrastive learning [8], [68]. These methods enable accuracy gains up to 15% in unseen noisy environments with SNR = 0 dB, whereas the transfer learning-based ODDA demonstrates classification improvements exceeding 12% for a miniaturized 23k parameters network. Moreover, although they take into consideration the available computational and memory resources, generic MCT networks are offline techniques and lack awareness of the deployment environment. In this work, server-side offline MCT represents the baseline on KWS noise robustness, acting as a starting point that we outperform through our environment-aware on-site adaptation.

C. ODL Frameworks

The ODL frameworks proposed in the literature aim to tackle the aforementioned costs only partially. Addressing memory constraints, Ren et al. [35] expanded a deployed, frozen DNN with a customizable layer, while [37] introduces the *Tiny Training Engine*, which enables sparse tensor update for both shallow and deep adaptation at the extreme edge. Nadalini et al. [38] proposed PULP-TrainLib, focusing on reducing the learning latency by leveraging Single Instruction Multiple Data (SIMD) support on the target architecture, as well as by parallelizing the computations over a multicore cluster. Jointly optimizing the memory requirements and the latency constraints, [78] and [79] relied on rematerialization, storing the activation of target designated layers as checkpoints and recomputing the remaining activations during the backward pass. We build our ODL methodology using the state-of-the-art TrainLib library [38], aware of the target architecture to leverage its available memory and storage, the structure of the memory hierarchy, and the computational units. We thus minimize the hardware-associated costs, while demonstrating the benefits of our ODL strategy in maximizing the KWS accuracy in real-world noisy conditions.

General-purpose ODL frameworks [38], [80] have been evaluated on MCUs for generic KWS tasks. Rusci and Tuytelaars [81] and Rusci et al. [82] proposed few-shot learning solutions for learning new target classes, while [83] introduced an online class incremental learning algorithm suitable for low-resource platforms. Nonetheless, such works fail to address the need for on-site domain adaptation and only consider clean utterances at test time. Conversely, on-device algorithms for adapting to background noises [19] or learning speech characteristics of novel speakers [84] have been proposed, yet these works only provide estimates regarding the ULP feasibility of their adaptation strategies. Compared to previous works, we propose an ODDA algorithm for robustifying KWS systems, tailored for ULP platforms.

TABLE II

OFF-THE-SHELF PLATFORMS EMPLOYED IN ODL. M0, M4, AND M7 REPRESENT ARM CORTEX-M0, ARM CORTEX-M4, AND ARM CORTEX-M7, RESPECTIVELY. LX106 REPRESENTS TENSILICA LX106. OUR WORK IS EVALUATED ON GAP9 AT BOTH 240 MHZ AND 370 MHZ, AND THE TABLE SHOWS THE MAXIMUM FREQUENCY TESTED

Reference	Device	SRAM	Flash
Ren *et al.* [35]	M4 @ 64 MHz	256 kB	1 MB
Lin *et al.* [37]	M7 @ 216 MHz	320 kB	1 MB
Patil *et al.* [78]	M0 @ 48 MHz	32 kB	256 kB
Khan *et al.* [79]	M4 @ 180 MHz	256 kB	2 MB
Pau *et al.* [85]	M7 @ 400 MHz	1 MB	2 MB
Kopparapu *et al.* [86]	M4 @ 64MHz	256 kB	1MB
Avi *et al.* [87]	M4 @ 84 MHz	96 kB	512 kB
Gimenez *et al.* [88]	M4 @ 64 MHz	256 kB	1 MB
Ficco *et al.* [89]	LX6 @ 240 MHz	520 kB	512 kB
Kang *et al.* [90]	GAP9 @ 200 MHz	1.6 MB	2 MB
Ours	GAP9 @ 370 MHz	1.6 MB	2 MB

Various off-the-shelf platforms have been employed in ODL in recent years, thanks to their cost-effectiveness, availability, and reusability [91], [92], as shown in Table II. Notably, GAP9 has an on-chip flash comparable in size to Arm Cortex-M7 [85] and Arm Cortex-M4 [79], while additionally featuring 1.6 MB SRAM. We thus demonstrate integration of our ODDA methodology in the GAP9 MCU, also considering the presence of the eight-core cluster and the accelerator operating at up to 370 MHz, which combine parallel programming features with ULP properties.

III. DOMAIN ADAPTATION METHODOLOGY

In this section, we describe the proposed ODDA methodology. We begin by describing the Noise-Aware KWS (NA-KWS) model that represents both the baseline for ODDA and its starting point, followed by presenting the target platform and the deployment process. We then introduce the ODL algorithm and its embodiment on the ULP platform. Lastly, we discuss the optimizations associated with performing domain adaptation on resource-constrained, extreme edge systems.

A. Noise-Aware KWS

KWS models that aim to specialize in on-site conditions reach higher accuracy if they were previously trained in an MCT setting. Given the diversity of conditions that can be present in an environment, for offline training, we consider noises from domestic sources, natural locations, offices, public spaces, and urban outdoor environments. We moreover consider *silence* (i.e., the absence of any background noise) as an additional setting, thus modeling quiet environments. By augmenting the clean utterances containing the target keywords with noises recorded in the wild, we thus produce a NA-KWS model. The training process completes after maximizing the accuracy on a joint validation set where all the training noises are present.

The NA-KWS methodology can be applied to any DNN topology to robustify it against additive background noise. In the current work, we consider DS-CNNs as our acoustic KWS models [39], [46], [54], contributing to a competitive system that can function in low-power and ULP platforms. The

TABLE III

DETAILED STRUCTURE OF THE `INT8`-QUANTIZED BACKBONE TOPOLOGY: LAYER TYPE (I.E., TOPOLOGY, KERNEL SIZE, STRIDE, AND PADDING PRECEDING THE LAYER), LAYER SHAPE, AND NUMBER OF PARAMETERS. EACH CONVOLUTIONAL LAYER IS FOLLOWED BY A BATCH NORMALIZATION LAYER AND A RELU ACTIVATION

Structure	DS-CNN Model S		M		L	
	Shape	# kPar.	Shape	# kPar.	Shape	# kPar.
2D Conv (10x4, s=2, p=[1, 1, 5, 5])	[64, 25, 5]	2.7	[172, 25, 5]	7.2	[276, 25, 5]	11.6
DW Conv (3x3, s=1, p=[1, 1, 1, 1])	[64, 25, 5]	0.7	[172, 25, 5]	1.9	[276, 25, 5]	3.0
PW Conv (1x1, s=1)	[64, 25, 5]	4.2	[172, 25, 5]	29.9	[276, 25, 5]	76.7
DW Conv (3x3, s=1, p=[1, 1, 1, 1])	[64, 25, 5]	0.7	[172, 25, 5]	1.9	[276, 25, 5]	3.0
PW Conv (1x1, s=1)	[64, 25, 5]	4.2	[172, 25, 5]	29.9	[276, 25, 5]	76.7
DW Conv (3x3, s=1, p=[1, 1, 1, 1])	[64, 25, 5]	0.7	[172, 25, 5]	1.9	[276, 25, 5]	3.0
PW Conv (1x1, s=1)	[64, 25, 5]	4.2	[172, 25, 5]	29.9	[276, 25, 5]	76.7
DW Conv (3x3, s=1, p=[1, 1, 1, 1])	[64, 25, 5]	0.7	[172, 25, 5]	1.9	[276, 25, 5]	3.0
PW Conv (1x1, s=1)	[64, 25, 5]	4.2	[172, 25, 5]	29.9	[276, 25, 5]	76.7
DW Conv (3x3, s=1, p=[1, 1, 1, 1])	-	-	-	-	[276, 25, 5]	3.0
PW Conv (1x1, s=1)	-	-	-	-	[276, 25, 5]	76.7
Avg Pool	[64]	-	[64]	-	[64]	-
Total # kPar.	**-**	**22.9**	**-**	**134.5**	**-**	**410.4**

DS-CNN is a family of models with a size ranging between 23k parameters and 411k parameters, as shown in Table III, which offers a wide complexity-accuracy trade-off, with a top-1 accuracy reported by Zhang et al. [46] exceeding 95% on the GSC noiseless 12-word task. DS-CNNs replace vanilla 2D convolution with paired depthwise and pointwise convolutions, which leads to a parameter reduction of $1/C_i + 1/k^2$, with C_i representing the number of input channels and k the convolutional kernel size.

B. Deployment

We consider the DNN model, which we deploy on the target platform, to be comprised of two components: a *frozen backbone* $f(\cdot)$, whose parameters will not change after deployment, and a *trainable unit* $g(\cdot)$ that is fine-tuned on-device. Without losing generality, we indicate with n the number of layers of a DNN, i the number of frozen layers, and $n-i$ the number of trainable layers, as shown in Algorithm 1. BatchNorm layers constitute an exceptional case, due to the presence of both trainable parameters (i.e., γ and β—weight and bias) and running statistics (i.e., mean and standard deviation, computed over the batch). Therefore, the running statistics can be optionally computed onboard even when the trainable parameters are frozen. The backbone f of the DNN projects an input in to a latent representation $y_i = f(\text{in})$, whereas the composite function $g \circ f$ of the trainable layers produces the output $y_n = g(y_i) = g(f(\text{in}))$. The frozen module is quantized to 8-bit (denoted as `int8` in the following) without loss of accuracy through PTQ (i.e., we denote y_j^{int} if $j \leq i$), whereas floating-point (`fp32`) representation (i.e., we denote y_j^{fp} if $j > i$) is employed for the trainable module to preserve numerical precision during on-device training.

We employ the RISC-V-based SoC introduced by Greenwaves Technologies as GAP9, which follows the PULP

Algorithm 1 ODDA Algorithm That Enables On-Site Noise Specialization for a DNN With i Frozen Layers and $n - i$ Trainable Layers

$noise \leftarrow record()$
for $ep = 1$ to N_{epochs} **do**
  for $k = 1$ to D **do**
    // *Preprocessing*
    $s_{aug} \leftarrow s_k + noise$
    $in \leftarrow \texttt{mfcc}(s_{aug})$
    // *Forward pass*
    $y_i^{int} \leftarrow f(in)$
    $y_i^{fp} \leftarrow \epsilon \cdot y_j^{int}$
    $y_n^{fp} \leftarrow g(y_i^{fp})$
    // *Loss computation*
    $L_{CE} \leftarrow \sum_{c=1}^{N_{cls}} label_k^c \cdot log(y_{n_c})$
    // *Backward pass*
    for $j = n - i + 1$ to n **do**
      $g_{y_{j-1}} \leftarrow \dfrac{\partial L_{CE}}{\partial y_j^{fp}} W_j$
      $g_{W_j} \leftarrow \dfrac{\partial L_{CE}}{\partial y_j^{fp}} y_{j-1}^{fp}$
      ▷// *Weight update*
      $W_j' \leftarrow W_j - \eta g_{W_j}$

architecture paradigm [93]. The SoC, shown in Fig. 2(a), consists of an eight-core cluster suitable for Digital Signal Processing (DSP) and Machine Learning (ML) workloads, with the parallelization controlled by a ninth core. The cluster domain also contains a CNN Neural Engine with 16 channels (NE16) for streaming 8-bit Multiply–Accumulate (MAC) operations. Both the accelerator and the cores share access to the 128 kB L1 Tightly Coupled Data Memory (TCDM) memory and to four Floating-Point Units (FPUs), enabling half (i.e., `fp16`) and single (i.e., `fp32`) precision operations.

The second component of the SoC is the Fabric Controller (FC) core, identical to the cluster cores, controlling the execution of the application and managing the peripherals through the micro-DMA. The third component is the micro-DMA, containing dedicated Direct Memory Access (DMA) channels between the L2 RAM (i.e., 1.5 MB) and the peripherals, which minimize the need for intervention by the processing cores. An additional L3 on-chip flash (i.e., 2 MB) is present in the SoC, while two external memory interfaces access an Octal-SPI 32 MB RAM and an Octal-SPI 64 MB flash. In the absence of a data cache, transactions between the three hierarchical memory levels are managed manually.

We deploy the frozen DS-CNN model using DORY code generator [31], after quantizing its corresponding parameters to `int8` using Quantlib [94]. We employ PULP-TrainLib [38] to generate the forward and backward passes for our retrainable layers (i.e., the last $n - i$ layers), as well as the loss computation code and the update of the weights.

C. On-Device Learning

Similar to the offline, server-side training, our proposed on-device, on-site adaptation methodology relies on supervised learning. During the adaptation stage, the additive nature of the noises enables us to alleviate the user from the tedious process of recording and labeling multiple noisy utterances. Instead, we first select a subset of clean, labeled utterances s_{dataset} of size D from the offline training set, and we store them on the target MCU for the ODL process. A continuous recording of on-site noises is then sufficient to capture the noise profile. The user initiates the recording process when the KWS system enters a previously unseen noisy environment.

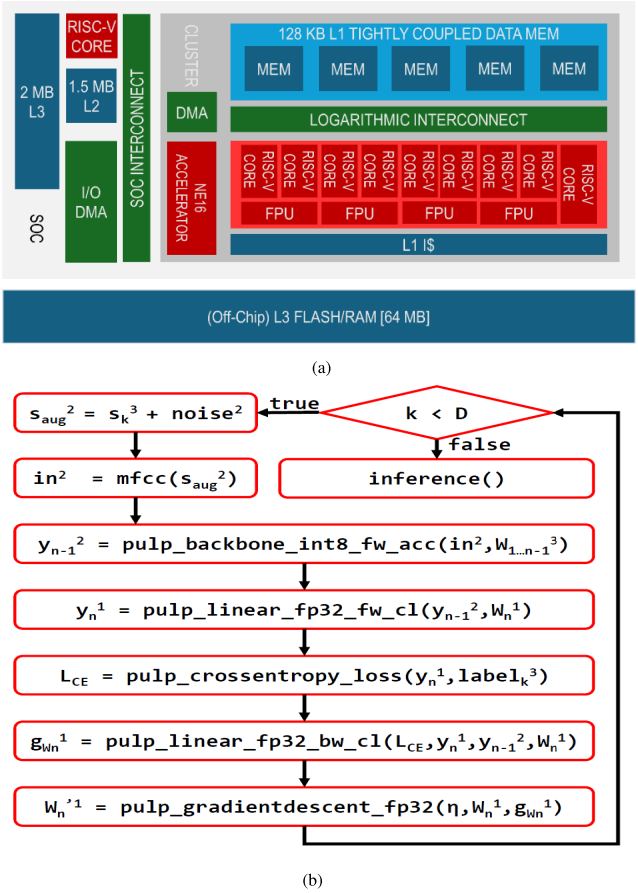


Fig. 2. Graphical depiction of ODDA. (a) GAP9 MCU, indicating the computational resources (i.e., FC core, multicore cluster, shared FPUs, and accelerator) and the memory levels. (b) ODDA embodied on GAP9 through the function calls of the `PULP-TrainLib` library [38], with only the classifier (i.e., $n - i = 1$) being trained on device. The superscripts (i.e., 1, 2, or 3) indicate the highest memory level where each variable resides. The name of each function contains the processing engine executing the function (i.e., `acc`elerator or `cl`uster) and the computing format (i.e., `fp32` or `int8`).

The ODL algorithm is shown in Algorithm 1, whereas in Fig. 2(b), we depict the mapping of Algorithm 1 on the GAP9 system assuming that only the classifier is trainable. The ODDA process initiates when novel on-site noise is available. Prerecorded, clean, and labeled utterances (i.e., a subset of the dataset used to train NA-KWS) are stored in the off-chip L3 memory. We begin by iterating for each s_k through the stored training dataset s_{dataset}, and we augment each sample with environmental noise recorded with the onboard microphone. We preprocess the noisy keyword by computing the spectrogram and extracting its Mel-Frequency Cepstral Coefficients (MFCCs), thus reducing the input size by a factor of 32 compared to the raw signal. The MFCCs are then fed to the frozen backbone.

The MFCCs are processed by the backbone in the forward pass (i.e., `pulp_backbone_int8_fw_acc`), which is offloaded on the computationally efficient NE16 accelerator. The backbone's frozen parameters $W_{1,\ldots,n-1}$ are stored in the on-chip L3 memory, the memory transfers to the L1 are orchestrated using the DMA to reduce stalling. The cores access data directly from and to the L1 TCDM, where temporary variables are stored, including the intermediate activations of each layer.

The `int8` output of the backbone y_i is dequantized (i.e., $y_i^{fp} \leftarrow \epsilon \cdot y_i^{\text{int}}$) and fed to the trainable layers $g(\cdot)$, thus producing the network output y_n. The classification result, together with the label of the augmented utterance, is used to compute the cross-entropy loss L_{CE} [i.e., `pulp_crossentropy_loss(y_n, label_k)` in Fig. 2(b)]. In the backward pass, the weight gradients (i.e., g_{W_j}—loss gradients with respect to the weights) are then calculated, for $j = n$ if only the classifier is trained. Through chain rule-based backpropagation, we identify the dependency on the intermediate features down to y_i^{fp}, as indicated in Algorithm 1. When training deeper layers, input gradients (i.e., $g_{y_{j-1}}$—loss gradients with respect to the input) are additionally computed and backpropagated to determine the update ratio of said layers. For the forward and backward pass of the trainable layer, we parallelize the computation on the multicore cluster, each core dispatching jobs over different FPUs. Lastly, we update the learnable parameters of the KWS and proceed to the next utterance.

Once the process is complete, the system reinitiates its always-on inference process with an expanded knowledge domain (i.e., the NA-KWS domains and the on-site novel environment). On the other hand, when the system is exposed to another novel environment, the adaptation process starts from the NA-KWS model, thus avoiding concept drifts and ensuring a noise-robust initialization checkpoint. The ODDA algorithm thus requires minimal effort from the end-user, while producing specialized KWS systems, ready to be utilized in noisy environments.

D. Resource-Constrained ODL

To achieve domain adaptation at the extreme edge for KWS systems, we propose an ODL algorithm that addresses all TinyML constraints (i.e., latency, energy consumption, storage, and memory). We target always-on, battery-powered embedded devices. Therefore, we first address energy consumption by considering lightweight models with a reduced number of MACs, in accordance with the energy efficiency of the platform. The computational effort associated with the ODDA process is the main contributor to the total energy cost of the system. The energy consumption scales linearly with the computational time, and we thus express the adaptation time t_{ODDA} as

$$t_{\text{ODDA}} \propto (t_{\text{fw}} + t_{\text{bw}}) \cdot \frac{S_{\text{dataset}}}{S_{\text{batch}}} \cdot N_{\text{epochs}} \tag{1}$$

where S_{dataset} represents the on-device training dataset size and S_{batch} represents the batch size. We represent as t_{fw} the inference latency for a batch of data (i.e., forward pass including the MFCC and backbone computation) and as t_{bw} the latency in computing the gradients (i.e., backward pass) and updating the model's parameters. The adaptation latency of ODDA further depends on the number of adaptation rounds (i.e., N_{epochs}).

The storage requirements refer to data read from the read-only, usually off-chip memory. That is the case for the clean utterances in s_{dataset}, stored to be augmented with on-site noise, as well as for the frozen parameters (i.e., $W_j, j \leq i$) of the deployed model. Conversely, the memory requirements concern the available read–write, on-chip memory. During the forward pass, intermediate activations (e.g., y_i in Algorithm 1) are temporarily stored in the memory. For the backward pass, as ODDA relies on GD learning, the inputs, parameters, activations, and weight and input gradients associated with a layer of index $j > i$ (i.e., y_{j-1}, W_j, y_j, g_{W_j}, and $g_{y_{j-1}}$, respectively) ought to be available simultaneously in the memory. Therefore, although the latency scales inversely with the batch size, as shown in (1), processing more samples in parallel is limited by the memory size containing the backpropagation-associated variables.

As mentioned in Section III-B, we allow for partially freezing the network, as having fewer trainable layers lowers the computational requirements t_{bw} of calculating the gradients and updating the weights. Simultaneously, by increasing the number of frozen layers and only retraining the classifier, fewer weights, activations, and gradients are employed in the adaptation phase, thus reducing the on-chip memory requirements. We further reduce the training latency by reducing S_{dataset} in (1), resulting in fewer Floating-Point Operations (FLOPs) and, therefore, in a faster learning process. Moreover, by storing only a fraction of keyword samples, we also lower our storage requirements.

IV. Experimental Results on Noise-Robust KWS

We consider the GSC [22] dataset as the primary scope of our KWS task. Our experiments target the 12-word problem, that is, ten keywords, together with *unknown*—any other word—and *silence*—the absence of any utterance. To increase the task difficulty when evaluating our methodology, we also consider the 35-word problem, comprised only of keywords. We use the training–validation–testing split proposed by the authors [22], which ensures no speaker contamination between the sets. We augment all samples with a random time shift between –200 and 200 ms.

To model the noisy environment, we consider additive noise sourced from real-world recordings of the DEMAND [57] dataset. To produce NA-KWS models, we use, with equal probability, 18 out of 19 noises (i.e., washroom, kitchen, living room, sports field, river, city park, small office, hallways, meeting rooms, subway station, cafeteria, restaurant, intersection, town square, cafe, and silence) to augment the samples. The omitted noise, herein named *target*, is present only after deployment, as opposed to the *source* noises used for offline MCT. All experiments are performed with a normalized SNR of 0 dB to ensure uniform evaluation conditions across all environments. The dataset management, our models, and the training and deployment pipeline will be open-sourced.[2]

[2]https://github.com/pulp-platform/odda-for-kws

TABLE IV
LIST OF HYPERPARAMETERS USED DURING NA-KWS PRETRAINING AND FOR ODDA

Hyperparameter	NA-KWS	ODDA
Epochs	≤ 60, patience=10	
Optimizer	Adam, $\beta = (0.9, 0.999)$	
Scheduler	LambdaLR, $\alpha = \begin{cases} 1, ep < 15 \\ 1/5, 15 \leq ep < 25 \\ 1/10, 25 \leq ep < 35 \\ 1/20, ep > 35 \end{cases}$	
Learning rate	10^{-3}	10^{-4}
Batch size	128	100-128

We evaluate our adaptation methodology in unbound storage and memory settings to determine the potential classification accuracy gain in unseen noise conditions. Unless stated otherwise, ODDA is performed for up to 60 epochs, with an early stopping patience of ten epochs. We set the batch size to 128 (or to the dataset size, when $s_{\text{dataset}} < 128$) and we employ the Adam optimizer, considering a step decay learning rate scheduler for an initial learning rate of 10^{-4}. The hyperparameters are summarized in Table IV. During the adaptation stage, we retrain all layers of the DS-CNN models and update the BatchNorm running mean and variance, employing all clean utterances used in the NA-KWS offline training stage.

A. Specializing on Unseen, On-Site Noise

In Table V, we analyze the ODDA impact given the task complexity and the model capacity, demonstrating that ODDA enables specialization on any previously unseen noise. We first evaluate ODDA on the traditional 12-word GSC. The impact of ODDA varies across target environments, with modest improvements of 1.15% on noises with stationary properties (i.e., *washing*), as the generally robust NA-KWS network is already capable of determining the speech features in the presence of these noises. On the other hand, domain adaptation is imperative for non-stationary noises, such as *meeting*. ODDA DS-CNN S outperforms by 12% its NA-KWS counterpart in these conditions.

On-site adaptation can, in specific cases, impact the performance more than the model complexity. Particularly, in the presence of unseen speech noise, a specialized DS-CNN S outperforms the 17× larger NA-KWS DS-CNN L by 8%. We demonstrate the positive ODDA impact on multiple model sizes, measuring improvements as great as 14% in *meeting* environments and top-1 accuracies up to 93% *washing* conditions. We attribute the results to the increased model capacity that improves the class separability in noisy conditions.

In Table V, we further evaluate ODDA given the number of keywords. When the task complexity increases to a 35-class problem, ODDA brings accuracy improvements between 1% (i.e., *washing*) and 8% (i.e., *meeting*) over NA-KWS models. Compared to the 12-class task, the increased complexity of the 35-word problem leads to both a reduced ODDA impact and lower absolute accuracies. We further define a 6-word task (i.e., "yes," "down," "left," "right," "off," and "stop") following [21], eliminating short, phonetically similar words (e.g., "no" and "go"). This leads to improvements up to 9% and unprecedented top-1 accuracies achieved after ODDA, ranging between 92.9% for *meeting* and 97.2% for *washing*.

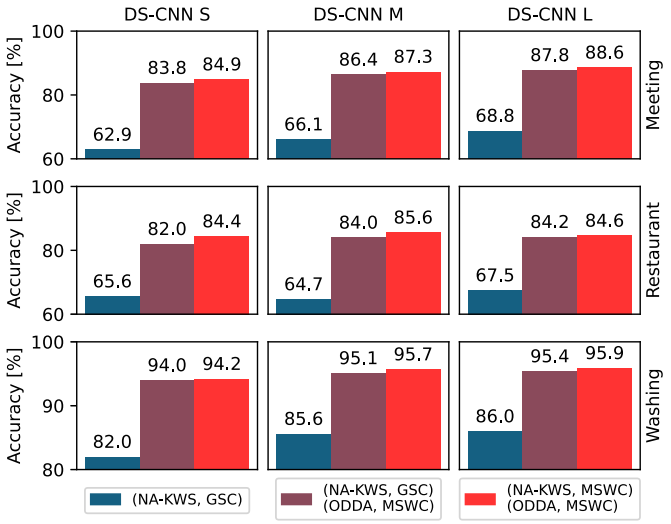


Fig. 3. Test accuracy [%] on the MSWC dataset, considering the NA-KWS pretraining dataset and the ODDA fine-tuning dataset, for DS-CNN models of different sizes.

B. Domain Adaptation in Coarticulated Keywords

In Fig. 3, we analyze the impact of ODDA on coarticulated keyword samples (i.e., words extracted automatically from a full, continuous sentence, as opposed to the manually recorded keywords of GSC). This adds to the complexity of performing KWS in unseen noisy environments. We employ the Multilingual Spoken Words Corpus (MSWC) [21] dataset, considering the 6-word problem and ensuring that only common words are accounted for in the experiment. For MSWC, we employ the training–validation–testing ratio and word frequency proposed by the authors in their `splits_en.csv` file,[3] without explicitly addressing the data imbalance.

We first pretrain a NA-KWS network on GSC data, and we evaluate it on MSWC data, depicted in blue in Fig. 3. The accuracy decreases by 11%–20% compared to evaluating said models on GSC data. We thus perform domain adaptation using clean utterances from the MSWC dataset, augmented with on-site noises [i.e., (NA-KWS GSC) and (ODDA MSWC)]. Our methodology improves the classification performance on noisy MSWC samples, the positive impact of ODDA increasing with the noise complexity (i.e., from 9.4% for *washing* to 21% for *meeting*). ODDA is particularly beneficial for smaller models (e.g., 9% for DS-CNN L and 14% for S in similar speech noise conditions), their reduced capacity making them more susceptible to domain shifts.

Thanks to ODDA, the accuracy after training offline and adapting on-site on disjoint datasets [i.e., (NA-KWS GSC) and (ODDA MSWC)] is comparable with the upper bound of performing both NA-KWS and ODDA on the target MSWC keywords, as the difference ranges between 0.2% and 2.4%. This demonstrates that on-site noise acquisition and offline

[3]https://storage.googleapis.com/public-datasets-mswc/splits/en.tar.gz

TABLE V

EFFECT ON THE ACCURACY [%] BETWEEN NA-KWS AND ODDA WHEN INCREASING THE TASK COMPLEXITY BY EXPANDING THE NUMBER OF CLASSES IN CONJUNCTION WITH AN INCREASE IN MODEL CAPACITY

Model	Noise Type → / Mode	Cafeteria 6 cls.	Cafeteria 12 cls.	Cafeteria 35 cls.	Meeting 6 cls.	Meeting 12 cls.	Meeting 35 cls.	Restaurant 6 cls.	Restaurant 12 cls.	Restaurant 35 cls.	Washing 6 cls.	Washing 12 cls.	Washing 35 cls.	Metro 6 cls.	Metro 12 cls.	Metro 35 cls.
S	NA-KWS	88.07	78.04	55.60	79.35	65.69	49.19	83.30	73.52	53.68	94.98	89.30	64.94	92.84	83.82	63.89
	ODDA	90.07	80.59	57.31	88.41	78.53	57.19	85.59	76.22	55.97	96.24	90.45	65.47	93.10	85.50	64.95
	Gain	+2.00	+2.55	+1.71	+9.06	+12.84	+8.00	+2.29	+2.70	+2.29	+1.26	+1.15	+0.53	+0.26	+1.68	+1.08
M	NA-KWS	90.29	82.60	59.60	83.18	69.96	51.06	85.61	77.83	57.03	96.63	91.64	67.58	94.30	87.10	65.82
	ODDA	91.78	84.25	60.69	91.41	84.23	57.87	88.76	80.12	59.01	97.10	92.86	68.15	94.15	89.51	66.66
	Gain	+1.49	+1.65	+1.09	+8.23	+14.27	+6.81	+3.15	+2.29	+1.98	+0.47	+1.22	+0.57	-0.15	+2.41	+0.84
L	NA-KWS	91.31	83.91	62.16	85.13	70.76	55.27	87.39	79.53	59.31	96.82	92.92	71.46	94.75	87.73	68.40
	ODDA	92.80	85.54	63.53	92.89	84.74	63.49	88.88	81.55	61.19	97.22	93.39	71.97	95.54	90.06	69.10
	Gain	+1.49	+1.63	+1.37	+7.76	+13.98	+8.22	+1.49	+2.02	+1.88	+0.40	+0.47	+0.51	+0.79	+2.33	+0.70

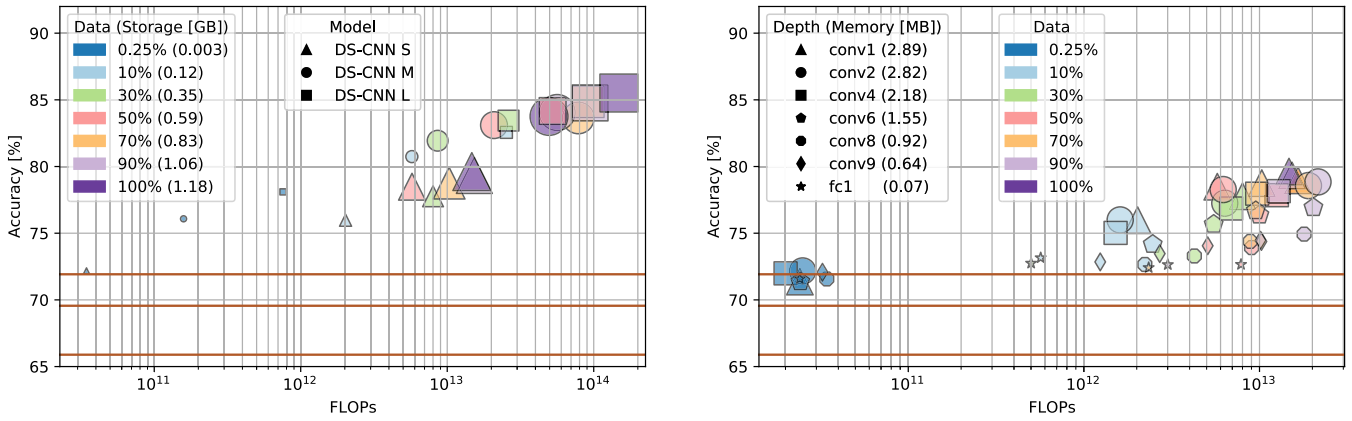


Fig. 4. Mitigating resource constraints during ODDA considering *speech* noise. Left: the storage cost given the DS-CNN model (i.e., *S*, *M*, and *L*) and the amount of randomly selected data used. Right: the memory cost given the number of updated layers of DS-CNN S, starting with the last linear (*fc1*) layer, for a batch size of two. The horizontal brown lines represent, starting with the lowest one, the baseline accuracy obtained with NA-KWS DS-CNN *S*, *M*, and *L*.

speech samples are sufficient to achieve superior noise robustness and satisfactory KWS accuracy in the presence of a speech domain gap [21].

V. ODL EVALUATION

In this section, we analyze ODDA in on-device, resource-constrained scenarios. We first explore the optimization techniques proposed in Section III-D, followed by evaluating platform-specific (i.e., GAP9 ULP MCU) optimization strategies. The section continues by analyzing the computational complexity, energy consumption, and energy efficiency of ODDA. Lastly, we evaluate the embodiment of ODDA, followed by discussing the limitations and future directions for ODDA.

A. Resource-Constrained ODDA

In Fig. 4 (left), we explore the storage-latency trade-offs for the optimization strategies proposed in Section III-D. We focus on speech noise scenarios (i.e., *meeting*), where ODDA is essential. We vary the number of GSC-12 samples stored (i.e., 32 kB of storage space per sample) for training, taking between 0.25% and 100% of the approximately 37 000 samples employed during NA-KWS. This does not include the validation set employed for early stopping, which is deemed optional, as our experiments show that ODDA applied for a fixed number of 20 training epochs outperforms NA-KWS models by 3% on average across five noises and all model sizes, comparable with the 4% increment of ODDA employed with early stopping.

With only 3 MB of data (i.e., 0.25% of the total), the adapted DS-CNN S (i.e., 26 kB) surpasses its NA-KWS counterpart by 6% and achieves comparable results with NA-KWS DS-CNN L. When 10% of the data becomes available (i.e., 120 MB—310 samples per class), ODDA DS-CNN S outperforms NA-KWS DS-CNN L by 10%. Employing the complete training set (i.e., 1.1 GB for 37 000 samples) generates accuracy gains up to 14%, demonstrating the benefits of on-site noise augmentation of prerecorded utterances in ODDA.

Notably, the adaptation depth also impacts the attainable accuracy. When comparing a classifier-only update with full retraining, we notice that fine-tuning the complete model could lead to overfitting when only 0.25% of the dataset is employed for adaptation. When 10% of the available data is used for on-device training, refining the entire DS-CNN S network outperforms by 1.2% a model of which only the classifier was adapted. In contrast, the difference increases to 6% for 100% of the training samples.

The model size and the amount of data used also impact the total number of FLOPs performed during ODDA. Models with larger capacity benefit from larger datasets available, but

they require up to 148.49 TFLOPs for DS-CNN L fine-tuned with 100% of the training samples. This is 73.5× more than adapting DS-CNN S using 10% of the data; therefore, although larger models achieve higher accuracies, their complexity increases the inference latency and slows adaptation.

To address the memory requirements, we evaluate the update of the last k layers of the model, instead of full-model fine-tuning. Simultaneously, we allow for the recomputation of the BatchNorm running statistics to better capture the distribution shift. To cope with the latency constraints, we opt for the smallest DS-CNN network and depict the results in Fig. 4 (right). Notably, we outperform the NA-KWS DS-CNN S baseline by 5% by refining only its last linear layer (i.e., *fc1*), achieving comparable accuracies with NA-KWS DS-CNN L. This requires storing 780 trainable parameters, the input and output activations, and the weight gradients, that is, only 10 kbyte of on-chip memory. The memory cost is, therefore, given by the computation of the running means and variance for the BatchNorm layers, that is, no more than 64 kB of on-chip memory.

B. Platform-Tailored ODDA

We target GAP9 ULP MCU for the on-device implementation of our proposed domain learning methodology. To this aim, we adapt the DS-CNN topology presented in Table III, optimizing it following the architecture of the target embedded system. First, we address the MFCC computation, reducing the computed coefficients per time window from 40 to 10. By reducing the number of filters, we thus reduce the granularity in the acoustic analysis instead of selecting a few coefficients after the computation, and thus focusing on the lower frequencies. Although dominated by the Fast Fourier Transform (FFT) computation, we reduce the preprocessing latency by 9.2% without accuracy loss.

Second, we aim to distribute the DNN processing over all available compute units, with the frozen `int8` backbone suitable for the NE16 accelerator. As asymmetric kernels are not supported for execution, we replace the 10×4 stem convolutional layer with a symmetric 3×3 one. Second, we consider a (1, 1, 1, 0) padding structure for all inputs of convolutional layers, thus obtaining a 51×12 input and progressively smaller intermediate feature maps. We thus reduce the number of model parameters by 8% and the number of operations by 20% for DS-CNN S. At the same time, the accuracy levels of the optimized models are comparable (i.e., 0.56% higher accuracy when averaging over silent and NA-KWS conditions) with the results obtained for the original topology described in Table III. We attribute this to the modified input structure and stem padding that emphasize temporal information.

Third, motivated by the results discussed in Section V-A, we implement an ODL split where we freeze the entire convolutional backbone and retrain only the fully connected classifier. In Table VI, we analyze the accuracy impact of additionally freezing the running statistics of the BatchNorm (i.e., mean and standard deviation), thus allowing for the normalization layers to be fused into the convolutional layer and thus reducing the number of FLOPs. We compare fine-tuning the trainable parameters of the entire network with fine-tuning only those of the classifier: the difference in accuracy ranges between 1.25% and 6.62%, generally larger when also freezing the running statistics, as the mean and variance partly capture the on-site data distribution. This is further visible when comparing a classifier-only training (i.e., accuracy gain does not exceed 4.33%) with additionally updating the running statistics within the backbone (i.e., accuracy increment of 7.46%).

TABLE VI

ACCURACY GAIN [%] FOR DIFFERENT AMOUNTS OF FINE-TUNING DATA, CONSIDERING THE UPDATE OF THE BATCHNORM RUNNING STATISTICS AND THE OPTIMIZER. FOR A COMPLETE UPDATE DEPTH, AN INITIAL LEARNING RATE OF 10^{-4} WAS EMPLOYED. FOR THE CLASSIFIER TRAINING, WE VARIED THE INITIAL LEARNING RATE BETWEEN 10^{-4} AND 10^{0} AND REPORTED THE BEST AVERAGE OVER THREE RUNS

Update depth	Running statistics	Optimizer	Adaptation data 100%	10%	0.25%
Complete	Updated	Adam	12.84	10.28	6.48
		SGD	4.88	2.95	1.87
	Frozen	Adam	9.73	5.75	1.70
		SGD	5.40	3.25	1.35
Classifier	Updated	Adam	6.95	7.46	5.68
		SGD	3.91	3.64	2.30
	Frozen	Adam	4.33	3.35	1.25
		SGD	3.35	2.70	1.49

In Table VI, we further investigate the impact of the optimizer. When the running statistics are computed exclusively offline, Adam-based learning generally outperforms SGD-based updates. Nevertheless, this comes at the cost of nine more FLOPs per parameter for the computation of the first and second moment and for the bias correction compared to the SGD-based learning, as well as 2× higher memory cost for storing the moments. To maximize computational efficiency and minimize the memory consumption, we opt for only fine-tuning the classifier, not updating the running statistics of the BatchNorm layers, and employing SGD as the optimizer for on-device training using 0.25% of the available dataset. In such a resource-constrained scenario tailored for extreme edge devices, ODDA still improves the classification accuracy by 1.49% compared to a pretrained network.

C. Adaptation Cost on the ULP Platform

In Table VII, we illustrate the impact on the forward pass of the `int8` backbone considering different hardware targets for the backbone computation. When parallelizing the computation from one to eight cores, the computational performance increases five times for DS-CNN S, from 0.8 MACs/cycle to 4 MACs/cycle. The pointwise convolution contributes the most (i.e., 6.2 MACs/cycle) to the speed-up due to its compute-dominated nature. The computational efficiency improves by 5.9× for DS-CNN L compared to DS-CNN S, as the 4.3× wider convolutions increase the time the network is compute-bound and reduce the relative contribution of I/O operations from 22% to 9%. When employing the hardware accelerator, an average computational efficiency of 15.67 MAC/cycle is measured for DS-CNN L considering its 2.66 Mcycles to complete the forward pass, compared to the 0.32 Mcycles for DS-CNN S.

In Fig. 5, we represent the number of cycles required to train the classifier, considering the four main stages involving

TABLE VII

LATENCY AND COMPUTATIONAL EFFICIENCY OF THE BACKBONE CONVOLUTIONAL LAYERS CONSIDERING DIFFERENT DEPLOYMENT TARGETS AND DS-CNN MODEL SIZES. $\sum$ INCLUDES ALL CONVOLUTIONAL LAYERS, WHEREAS *Total* INCLUDES $\sum$, THE AVERAGE POOLING LAYER, AND THE I/O OPERATIONS

Processing unit →		Single core		8-core cluster		NE16 accelerator	
Model size	Conv. layer	Cycles [M]	MACs/ cycle	Cycles [M]	MACs/ cycle	Cycles [M]	MACs/ cycle
S	2D	0.23	0.30	0.039	1.76	0.039	1.77
	DW	0.92	0.27	0.19	1.30	0.08	3.06
	PW	1.44	1.23	0.28	6.20	0.064	27.37
	$\sum$	2.59	0.80	0.51	4.04	0.18	11.28
	Total	2.72	0.76	0.65	3.20	**0.32**	**6.49**
M	2D	0.54	0.34	0.084	2.18	0.084	2.19
	DW	2.44	0.27	0.49	1.36	0.18	3.68
	PW	8.20	1.55	1.41	9.00	0.34	37.17
	$\sum$	11.19	1.21	1.99	6.83	0.61	22.31
	Total	11.61	1.17	2.41	5.63	**1.04**	**13.08**
L	2D	0.83	0.36	0.13	2.32	0.13	2.32
	DW	4.55	0.29	0.71	1.83	0.35	3.71
	PW	24.63	1.62	4.22	9.47	1.08	37.08
	$\sum$	30.01	1.39	5.06	8.22	1.56	26.69
	Total	31.07	1.34	6.12	6.79	**2.66**	**15.67**

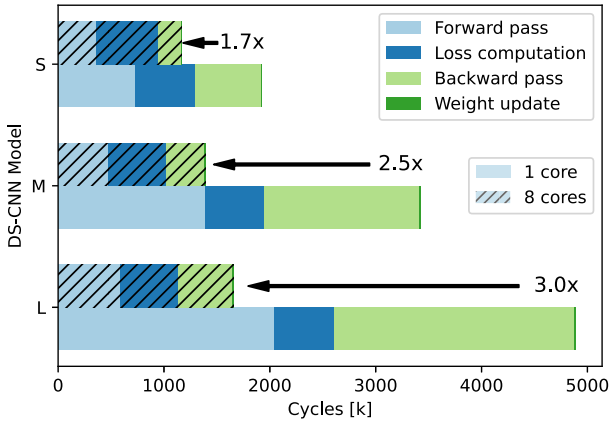


Fig. 5. Breakdown of computation time during fine-tuning stages for the classifier, considering different deployment targets and model sizes, for a batch size of 128 samples.

the linear layer: forward pass, cross-entropy loss computation, backward pass, and weights update (i.e., performed after exhausting a batch of 128 samples). The `fp32` classifier is analyzed in a single- and multicore setting, given the size of the tensors produced by the DS-CNN backbones. The loss computation, independent of the network size and with limited parallelization potential, averages 4.4 kcycle per sample. Considering that we are only updating the last layer, during the backward pass we are only computing the weight gradient, but not the input gradient. As the backbone increases, so does the size of the classifier's input, and the contribution of the compute-bound backward pass to the latency increases from 33% to 46%, dominating the loss computation. Consequently, the positive impact of parallelization increases with the size of the classifier, from a 1.65× improvement in computational effort (i.e., 2.4× when excluding the loss computation) for DS-CNN S to 2.95× (i.e., 3.9× when excluding the loss computation) for DS-CNN L, as shown in Fig. 5.

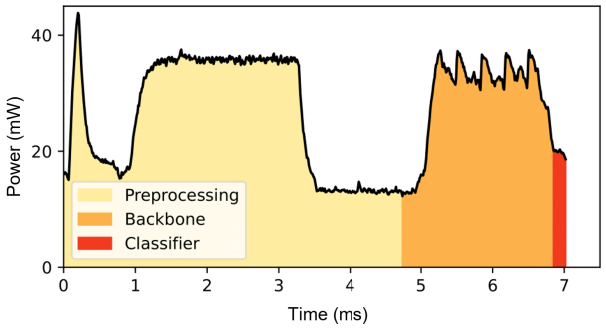


Fig. 6. Power trace of ODDA for DS-CNN *S* implemented on GAP9 operating at 240 MHz and 650 mV.

D. Energy Efficiency in ODDA

In Table VIII, we depict the average power and total energy consumption of the main components of our KWS pipeline, that is, the preprocessing stage computing `fp16` MFCCs, the `int8` forward-only backbone, and the training of the `fp32` classifier. We consider two operating conditions: Low-Power Mode (LPM) (i.e., 240 MHz, 650 mV) and High-Performance Mode (HPM) (i.e., 370 MHz, 800 mV).

Computing and scaling the coefficients that are fed into the network requires between 115 μJ over 4.73 ms in LPM and 183 μJ over 3.10 ms in HPM. One sample is shown here in Fig. 6, for a DS-CNN S backbone which extracts features using the NE16 accelerator, while the classifier retraining is distributed to the eight-core cluster. Note that the depicted preprocessing stage contains the movement of the clean utterance from the L3 memory to the L1 level, its noise augmentation, the MFCC computation, and the quantization of the coefficients from `fp16` to `int8`. The preprocessing step is comparable to or even exceeds the latency in obtaining the intermediate representation generated by the backbone when the networks employed are small and the DNN computation is accelerated on the neural engine. However, this preprocessing could be performed as a one-off process per sample during the first training epoch, with limited impact on the global latency or energy consumption over several epochs.

The results presented in Table VIII confirm that the benefits of larger networks are emphasized by using more efficient computational resources. Although the average power consumption increases when employing more computational resources (i.e., more cores) or more complex processing engines (i.e., NE16 accelerator), the reductions in latency generate energy savings. Specifically, for DS-CNN S backbone inference performed on GAP9 operating in LPM, the average power increases by 1.6× when the entire cluster is used compared to employing a single core, yet the energy cost reduces to a third. Similarly, a 1.8× increase in power consumption when offloading the backbone computation to the accelerator generates a 5.4× reduction in total energy consumption. As the networks increase in size and the MAC operations dominate memory transfers, it becomes 8.2× more energy efficient to employ the accelerator over a single core (and 2.9× more efficient over the use of the eight-core cluster) in LPM.

In scenarios that require minimal latency, operating in HPM enables the KWS system to offer a prediction in only 4.43 ms when offloading the DS-CNN S backbone to the NE16

TABLE VIII
ODL COSTS CONSIDERING LPM AND HPM OPERATING CONDITIONS, FOR MODELS OF DIFFERENT SIZES

Operating conditions			240 MHz, 650 mV			370 MHz, 800 mV		
Model	Stage	Unit	Latency [ms]	Power [mW]	Energy [μJ]	Latency [ms]	Power [mW]	Energy [μJ]
Preprocessing			4.73	24.40	115	3.10	58.98	183
DS-CNN S	Backbone	NE16 accelerator	1.94	31.83	62	1.29	67.47	87
		8-core cluster	3.84	27.80	107	2.52	58.44	147
		Single-core	19.27	17.34	334	12.53	34.21	429
	Classifier	8-core cluster	0.07	20.07	1	0.04	40.71	2
		Single-core	0.14	17.44	3	0.10	34.61	3
DS-CNN M	Backbone	NE16 accelerator	4.51	35.90	162	2.66	81.41	217
		8-core cluster	15.04	32.70	492	9.20	78.01	717
		Single-core	77.58	17.57	1363	50.13	35.55	1789
	Classifier	8-core cluster	0.10	23.13	2	0.07	50.78	3
		Single-core	0.30	17.67	5	0.19	35.31	7
DS-CNN L	Backbone	NE16 accelerator	10.15	40.59	412	6.91	83.93	580
		8-core cluster	33.24	34.72	1154	21.21	86.74	1840
		Single-core	189.63	17.77	3370	122.70	36.04	4424
	Classifier	8-core cluster	0.12	25.51	3	0.07	56.03	4
		Single-core	0.44	17.89	8	0.28	36.04	10

TABLE IX
EDP [ms·mj] FOR THE FORWARD PASS OF THREE DS-CNN-BASED KWS PIPELINES DEPLOYED ON GAP9 OPERATING IN LPM AND HPM MODES. FOR REFERENCE, WE SHOW THE EDP REPORTED BY [95] WITH FOUR DS-CNN CONFIGURATIONS (I.E., NAMED AFTER THEIR INPUT SIZE) ON THREE STM32 MCUS. NOTE THAT THE DS-CNN MODELS PROPOSED BY [95] HAVE 46.5K PARAMETERS, COMPARABLE TO OUR 22.9K PARAMETERS DS-CNN *S*

DS-CNN	U5	H7	N7	DS-CNN	LPM	HPM
15x32	75.96	83.00	3.48	S	0.67	0.69
30x32	211.20	243.38	5.84	M	1.30	1.17
15x63	174.60	168.98	3.54	L	4.81	4.64
30x63	653.45	672.10	7.56			

accelerator. Furthermore, the larger backbone can be utilized for a total of 10.08 ms to improve the classification accuracy while maintaining real-time capabilities.

Bartoli et al. [96] introduced Energy-Delay Product (EDP) as a metric to evaluate energy efficiency. Capturing the trade-off between energy consumption and latency by measuring their product, EDP is suitable to assess the degree to which an ML pipeline efficiently utilizes the architecture of the target extreme edge device. In Table IX, we show the EDP for the forward pass of our KWS pipeline for three DS-CNN networks in LPM and HPM. As a comparison, we consider the KWS pipeline developed by [95], relying on four DS-CNN configurations (i.e., 46.5k parameter) and deployed on STM32 MCUs: N6 (Cortex-M55), H7 (Cortex-M7), and U5 (Cortex-M33). With an EDP of 0.67 ms·mJ (i.e., 5.2× more efficient than the N6 deployment by [95]), our DS-CNN S-based pipeline demonstrates both the energy efficiency of the deployment platform and the optimal tailoring of our pipeline to the available resources. Our DS-CNN L-based inference is comparable with the related work at 8.8× more parameters, a larger capacity (and thus better classification performance) for the same energy cost.

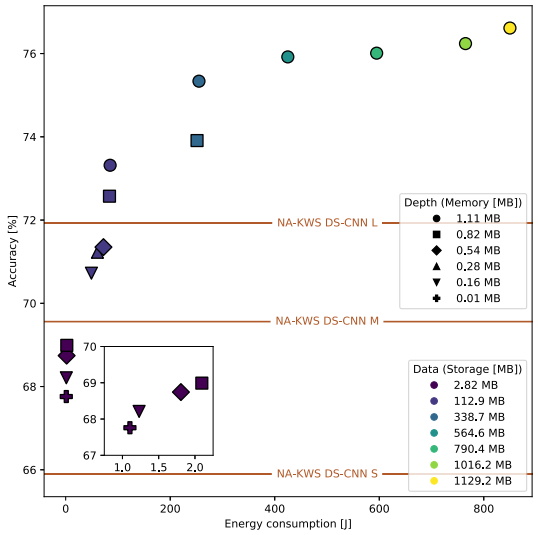


Fig. 7. Pareto front of energy consumption for ODDA for DS-CNN S, considering GAP9 MCU that operates in LPM. The number of clean utterances in the read-only memory determines the storage requirements, whereas the adaptation depth and the tensors stored in the read-write memory determine the memory requirements.

In Fig. 7, we extend our energy consumption analysis to include different adaptation depths and various on-device dataset sizes, considering the DS-CNN S network. We explicitly account for the preprocessing cost, as well as for the energy efficiency of offloading the backbone to the NE16 accelerator, and the `fp32` trainable layers to the multicore cluster. Notably, all DS-CNN S networks outperform the 65.7% accuracy of the NA-KWS after adaptation.

In the Pareto front, we first analyze the update of the classifier; its memory requirements of 10 kB fit within the L1 TCDM memory. 3 MB of storage are sufficient to reach

67.8% accuracy, outperforming the NA-KWS DS-CNN *S* by 2% in an extreme edge scenario. Increasing the size of the off-chip memory to 112 Mbyte, we obtain the accuracy of 68.8%.

The remaining Pareto-optimal points are obtained after expanding the on-chip memory directly accessible by the processing engines. The update can thus include deeper and wider layers beyond the 10 kB required to fine-tune the classifier. By fine-tuning all but the last three convolutional layers (i.e., 0.82 MB), the accuracy continues to improve, reaching 72.6% with 72.25 J for the entire adaptation routine, outperforming the NA-KWS DS-CNN L. Finally, ODDA can be applied not only in extreme edge scenarios on ULP platforms, but also in edge settings, where 850 J are available, as training all but the first layer of DS-CNN S benefiting from all clean utterances available can reach an accuracy of up to 76.6%.

E. Odda Embodied on GAP9 MCU

We integrate all findings discussed to devise an on-device training scenario using ten samples per class (i.e., 100 samples), requiring only 3 MB of storage. The memory requirements for the trainable classifier (i.e., initial weights, intermediate activations and the output, weights gradients, and updated weights) amount to only 10 kB of L1 memory, for one sample per batch. 20 training epochs are sufficient for DS-CNN S to outperform its NA-KWS counterpart. In a LPM scenario, the total training time is 13.5 s, for an average power consumption of 26.5 mW (i.e., a modest energy consumption of 357 mJ). Considering 3.7 V supplied by a 100 mAh battery, adequate for IoT systems [97], we can compute the device lifetime. After training against the new noise, the system switches to the inference mode. Considering ten inference rounds per second [46], an always-on, battery-powered GAP9 can perform KWS inference for 27.5 h. This analysis demonstrates that we achieved noise-robust KWS systems through ODL on ULP platforms.

As seen in Table VIII, the latency of the classifier is two orders of magnitude lower than that of the backbone, whereas the energy consumption is between 43 and 145 times lower, due to its reduced complexity. This favors ODL approaches that require multiple training epochs: the backbone features can be computed once per sample, stored in the L2 memory, and used iteratively to update the classifier. Considering a small KWS system trained for the 12-word problem, learning a new noise in ten epochs with ten samples per class would require only 7.5 kB of L2 memory. Most importantly, the energy cost in LPM would be reduced to 22.4 mJ, 9.5× less than employing the entire pipeline for each noisy sample.

VI. Limitations and Future Directions

ODDA is explicitly designed for linear distortions produced by additive background noise, achieving accuracy improvements up to 21% in unseen noise conditions. Our methodology does not address reverberations or microphone-specific distortions, nor does it consider the speech characteristics of the speaker (e.g., gender, accent, and prosody), which could be addressed through MCT or on-device augmentation. Regarding the loudness of the environment, the current work considers SNR levels of 0 dB, whereas future work includes evaluating ODDA given SNR levels between 30 and −10 dB [70].

Our methodology splits the network into a frozen block and a trainable one, only updating the latter through backpropagation. Instead, sparsely updating the deployed model [37], [98] (i.e., training a subset of each layer's parameters) could enable deeper updates and thus potentially bring accuracy improvements. ODDA could also leverage efficient memory management strategies to reduce the energy consumption throughout the learning process, while additionally addressing the reduced on-chip memory.

Rematerialization [78] and checkpointing [79] have been recently introduced, proposing to store only certain intermediate activations produced during the forward pass and to recompute the remaining ones during the backpropagation step.

Our methodology relies on `fp32` to represent the weights and activations of the layers that are trained on-device. To additionally minimize the memory footprint during backpropagation, Nadalini et al. [99] employed `fp16` representation for the trainable layers, whereas Lin et al. [37] proposed Quantization-Aware Scaling (QAS) to enable 8-bit quantized training. As ODDA employs Post-Training Quantization (PTQ) for the `int8` backbone and represents the trainable layers in `fp32` format, it could further benefit from structured pruning [100] or adaptive model compression techniques [101], [102], [103] to reduce its memory footprint.

Lastly, ODDA is a local learning technique, that is, the entire pipeline from data acquisition to inference and training takes place on a single platform near the sensor. To increase the access to on-site data, ODDA could be integrated into an Federated Learning (FL) (i.e., each device trains its model on its local data, sharing the resulting model) [86], [88], [104], [105], [106], whereas Split Learning (SL) (i.e., each device trains a subset of the layers and the intermediate activations are shared) [107], [108], [109] could be used to share the computational cost across multiple devices.

VII. Conclusion

This work presents ODDA, an ODL methodology enabling KWS to adapt to previously unseen on-site noises. We evaluated our methodology on the KWS dataset GSC and we measured accuracy improvements up to 14% in the presence of non-stationary speech noise, and up to 21% for coarticulated keywords. We tailored our ODL methodology for deployment on extreme edge devices, and we evaluated its suitability on the GAP9 ULP MCU. We demonstrated that only 3 MB of training data is needed for the DS-CNN S model to surpass its NA-KWS noise-robust counterpart by 5%. When only the classifier is fine-tuned, no more than 10 kB of on-chip memory is required for the training stage. With an average power consumption of 26 mW for ODL, an adaptation process completes in less than 14 s and enables the battery-powered, always-on KWS system to produce robust predictions in noisy environments for over 27 h.

References

[1] I. López-Espejo, Z.-H. Tan, J. H. L. Hansen, and J. Jensen, "Deep spoken keyword spotting: An overview," *IEEE Access*, vol. 10, pp. 4169–4199, 2022.

[2] C. Cioflan, L. Cavigelli, M. Rusci, M. de Prado, and L. Benini, "On-device domain learning for keyword spotting on low-power extreme edge embedded systems," in *Proc. IEEE 6th Int. Conf. AI Circuits Syst. (AICAS)*, Apr. 2024, pp. 6–10.

[3] M. Jung, Y. Jung, J. Goo, and H. Kim, "Multi-task network for noise-robust keyword spotting and speaker verification using CTC-based soft VAD and global query attention," in *Proc. Interspeech*, 2020, pp. 931–935.

[4] J. Casebeer, J. Wu, and P. Smaragdis, "Meta-AF echo cancellation for improved keyword spotting," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Apr. 2024, pp. 676–680.

[5] C. Yang, Y. M. Saidutta, R. S. Srinivasa, C.-H. Lee, Y. Shen, and H. Jin, "Robust keyword spotting for noisy environments by leveraging speech enhancement and speech presence probability," in *Proc. Interspeech*, Aug. 2023, pp. 1638–1642.

[6] I. López-Espejo, Z.-H. Tan, and J. Jensen, "A novel loss function and training strategy for noise-robust keyword spotting," *IEEE/ACM Trans. Audio, Speech, Language Process.*, vol. 29, pp. 2254–2266, 2021.

[7] D. Ng, Y. Chen, B. Tian, Q. Fu, and E. S. Chng, "Convmixer: Feature interactive convolution with curriculum learning for small footprint and noisy far-field keyword spotting," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2022, pp. 3603–3607.

[8] D. Ng et al., "I2CR: Improving noise robustness on keyword spotting using inter-intra contrastive regularization," in *Proc. Asia–Pacific Signal Inf. Process. Assoc. Annu. Summit Conf. (APSIPA ASC)*, 2022, pp. 605–611.

[9] D. Ng et al., "Small footprint multi-channel network for keyword spotting with centroid based awareness," in *Proc. Interspeech*, 2023, pp. 296–300.

[10] M. Scherer, C. Cioflan, M. Magno, and L. Benini, "Work in progress: Linear transformers for TinyML," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Benin, Mar. 2024, pp. 1–2.

[11] G. Chen, C. Parada, and G. Heigold, "Small-footprint keyword spotting using deep neural networks," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2014, pp. 4087–4091.

[12] M. Sun et al., "Compressed time delay neural network for small-footprint keyword spotting," in *Proc. Interspeech*, Aug. 2017, pp. 3607–3611.

[13] S. Sigtia, R. Haynes, H. Richards, E. Marchi, and J. S. Bridle, "Efficient voice trigger detection for low resource hardware," in *Proc. Interspeech*, 2018, pp. 2092–2096.

[14] H. Liu, T. Fan, and P. Wu, "Audio-visual keyword spotting based on adaptive decision fusion under noisy conditions for human–robot interaction," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2014, pp. 6644–6651.

[15] B. Rehman, R. Yagfarov, and A. Klimchik, "Interactive mobile robot in a dynamic environment," *IFAC-PapersOnLine*, vol. 51, no. 30, pp. 354–359, 2018.

[16] Y. Na, Z. Wang, L. Wang, and Q. Fu, "Joint ego-noise suppression and keyword spotting on sweeping robots," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2022, pp. 7547–7551.

[17] Y. Gong, Y.-A. Chung, and J. Glass, "AST: Audio spectrogram transformer," in *Proc. Interspeech*, 2021, pp. 571–575.

[18] A. Berg, M. O'Connor, and M. T. Cruz, "Keyword transformer: A self-attention model for keyword spotting," in *Proc. Interspeech*, Aug. 2021, pp. 4249–4253.

[19] C. Cioflan, L. Cavigelli, M. Rusci, M. De Prado, and L. Benini, "Towards on-device domain adaptation for noise-robust keyword spotting," in *Proc. IEEE 4th Int. Conf. Artif. Intell. Circuits Syst. (AICAS)*, Jun. 2022, pp. 82–85.

[20] V.-V. Eklund, A. Diment, and T. Virtanen, "Noise, device and room robustness methods for pronunciation error detection," in *Proc. 30th Eur. Signal Process. Conf. (EUSIPCO)*, Aug. 2022, pp. 140–144.

[21] M. Mazumder et al., "Multilingual spoken words corpus," in *Proc. Neural Inf. Process. Syst. Track Datasets Benchmarks*, 2021.

[22] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," 2018, *arXiv:1804.03209*.

[23] A. Mohanty, A. Frischknecht, C. Gerum, and O. Bringmann, "Behavior of keyword spotting networks under noisy conditions," in *Int. Conf. Artif. Neural Netw.*, 2021, pp. 369–378.

[24] H. Park, P. Zhu, I. L. Moreno, and N. Subrahmanya, "Noisy student-teacher training for robust keyword spotting," in *Proc. Interspeech*, 2021, pp. 331–335.

[25] Y. Huang, T. Hughes, T. Z. Shabestary, and T. Applebaum, "Supervised noise reduction for multichannel keyword spotting," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Apr. 2018, pp. 5474–5478.

[26] Y. A. Huang, T. Z. Shabestary, and A. Gruenstein, "Hotword cleaner: Dual-microphone adaptive noise cancellation with deferred filter coefficients for robust keyword spotting," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2019, pp. 6346–6350.

[27] M. Rusci, M. Fariselli, M. Croome, F. Paci, and E. Flamand, "Accelerating RNN-based speech enhancement on a multi-core MCU with mixed FP16-INT8 post-training quantization," in *Machine Learning and Principles and Practice of Knowledge Discovery in Databases*. Cham, Switzerland: Springer, 2023, pp. 606–617.

[28] C. Banbury et al., "MLPerf tiny benchmark," in *Proc. Neural Inf. Process. Syst. Track Datasets Benchmarks*, 2021.

[29] Q. Zhou et al., "On-device learning systems for edge intelligence: A software and hardware synergy perspective," *IEEE Internet Things J.*, vol. 8, no. 15, pp. 11916–11934, Aug. 2021.

[30] T. Chen et al., "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proc. 13th USENIX Conf. Operating Syst. Design Implement.*, 2018, pp. 578–594.

[31] A. Burrello, A. Garofalo, N. Bruschi, G. Tagliavini, D. Rossi, and F. Conti, "DORY: Automatic end-to-end deployment of real-world DNNs on low-cost IoT MCUs," *IEEE Trans. Comput.*, vol. 70, no. 8, pp. 1253–1268, Aug. 2021.

[32] R. David et al., "Tensorflow lite micro: Embedded machine learning for tinyml systems," in *Proc. Mach. Learn. Syst.*, 2021, pp. 800–811.

[33] L. Turchet, G. Fazekas, M. Lagrange, H. S. Ghadikolaei, and C. Fischione, "The Internet of Audio Things: State of the art, vision, and challenges," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 10233–10249, Oct. 2020.

[34] H. Cai, C. Gan, L. Zhu, and S. Han, "TinyTL: Reduce memory, not parameters for efficient on-device learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 11285–11297.

[35] H. Ren, D. Anicic, and T. A. Runkler, "TinyOL: TinyML with online-learning on microcontrollers," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2021, pp. 1–8.

[36] B. Sudharsan, J. G. Breslin, and M. I. Ali, "ML-MCU: A framework to train ML classifiers on MCU-based IoT edge devices," *IEEE Internet Things J.*, vol. 9, no. 16, pp. 15007–15017, Aug. 2022.

[37] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, C. Gan, and S. Han, "On-device training under 256KB memory," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 22941–22954.

[38] D. Nadalini, M. Rusci, G. Tagliavini, L. Ravaglia, L. Benini, and F. Conti, "PULP-TrainLib: Enabling on-device training for RISC-V multi-core MCUs through performance-driven autotuning," in *Proc. Embedded Comput. Syst., Archit., Model., Simul.*, 2022, pp. 200–216.

[39] P. M. Sørensen, B. Epp, and T. May, "A depthwise separable convolutional neural network for keyword spotting on an embedded system," *EURASIP J. Audio, Speech, Music Process.*, vol. 2020, no. 1, pp. 1–14, Dec. 2020.

[40] B. Liu et al., "A 3.8-μW 10-keyword noise-robust keyword spotting processor using symmetric compressed ternary-weight neural networks," *IEEE Open J. Solid-State Circuits Soc.*, vol. 3, pp. 185–196, 2023.

[41] S. Zhou et al., "An 8.62-μW 75-dB DR SoC fully integrated SoC for spoken language understanding," *IEEE J. Solid-State Circuits*, vol. 60, no. 11, pp. 4002–4017, Nov. 2025.

[42] S. Y. A. Yarga and S. U. N. Wood, "Neuromorphic keyword spotting with pulse density modulation MEMS microphones," in *Proc. Interspeech*, 2024, pp. 3275–3279.

[43] X. Ji, Y. Chen, J. Wang, G. Zhou, C. Sing Lai, and Z. Dong, "Time-frequency hybrid neuromorphic computing architecture development for battery state-of-health estimation," *IEEE Internet Things J.*, vol. 11, no. 24, pp. 39941–39957, Dec. 2024.

[44] X. Ji, Z. Dong, L. Zhu, C. Hu, and C. S. Lai, "An efficient human activity recognition in-memory computing architecture development for healthcare monitoring," *IEEE J. Biomed. Health Informat.*, vol. 30, no. 1, pp. 1–14, Jan. 2026.

[45] S. Fernández, A. Graves, and J. Schmidhuber, "An application of recurrent neural networks to discriminative keyword spotting," in *Proc. ICANN*, 2007, pp. 220–229.

[46] Y. Zhang, N. Suda, L. Lai, and V. Chandra, "Hello edge: Keyword spotting on microcontrollers," 2017, *arXiv:1711.07128*.

[47] S. Ö. Arık et al., "Convolutional recurrent neural networks for small-footprint keyword spotting," in *Proc. Interspeech*, 2017, pp. 1606–1610.

[48] Y. Wei, Z. Gong, S. Yang, K. Ye, and Y. Wen, "EdgeCRNN: An edge-computing oriented model of acoustic feature enhancement for keyword spotting," *J. Ambient Intell. Humanized Comput.*, vol. 13, no. 3, pp. 1–11, Mar. 2022.

[49] C. Banbury et al., "MicroNets: Neural network architectures for deploying TinyML applications on commodity microcontrollers," in *Proc. Int. Conf. Mach. Learn. Syst.*, 2021, pp. 517–532.

[50] D. Peter, W. Roth, and F. Pernkopf, "End-to-end keyword spotting using neural architecture search and quantization," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2022, pp. 3423–3427.

[51] D. Coimbra de Andrade, S. Leo, M. Loesener Da Silva Viana, and C. Bernkopf, "A neural attention model for speech command recognition," 2018, *arXiv:1808.08929*.

[52] Y. Xi, H. Li, B. Yang, H. Li, H. Xu, and K. Yu, "TDT-KWS: Fast and accurate keyword spotting using token-and-duration transducer," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Apr. 2024, pp. 11351–11355.

[53] K. Chen, X. Du, B. Zhu, Z. Ma, T. Berg-Kirkpatrick, and S. Dubnov, "HTS-AT: A hierarchical token-semantic audio transformer for sound classification and detection," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2022, pp. 646–650.

[54] Y. Lu, W. Shan, and J. Xu, "A depthwise separable convolution neural network for small-footprint keyword spotting using approximate MAC unit and streaming convolution reuse," in *Proc. IEEE Asia Pacific Conf. Circuits Syst. (APCCAS)*, Nov. 2019, pp. 309–312.

[55] C. Valentini-Botinhao and J. Yamagishi, "Speech enhancement of noisy and reverberant speech for text-to-speech," *IEEE/ACM Trans. Audio, Speech, Language Process.*, vol. 26, no. 8, pp. 1420–1433, Aug. 2018.

[56] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "Librispeech: An ASR corpus based on public domain audio books," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Apr. 2015, pp. 5206–5210.

[57] J. Thiemann, N. Ito, and E. Vincent, "Demand: A collection of multichannel recordings of acoustic noise in diverse environments," in *Proc. Meetings Acoust.*, 2013, pp. 1–6.

[58] D. Snyder, G. Chen, and D. Povey, "MUSAN: A music, speech, and noise corpus," 2015, *arXiv:1510.08484*.

[59] H. Chen et al., "The first multimodal information based speech processing (Misp) challenge: Data, tasks, baselines and results," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2022, pp. 9266–9270.

[60] R. Cutler et al., "ICASSP 2022 acoustic echo cancellation challenge," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2022, pp. 9107–9111.

[61] D. Pearce and H.-G. Hirsch, "The aurora experimental framework for the performance evaluation of speech recognition systems under noisy conditions," in *Proc. 6th Int. Conf. Spoken Lang. Process. (ICSLP)*, Oct. 2000, pp. 29–32.

[62] E. Fonseca, X. Favory, J. Pons, F. Font, and X. Serra, "FSD50K: An open dataset of human-labeled sound events," *IEEE/ACM Trans. Audio, Speech, Language Process.*, vol. 30, pp. 829–852, 2022.

[63] G. Wichern et al., "WHAM!: Extending speech separation to noisy environments," in *Proc. Interspeech*, Sep. 2019, pp. 1368–1372.

[64] J. Barker, R. Marxer, E. Vincent, and S. Watanabe, "The third 'CHiME' speech separation and recognition challenge: Dataset, task and baselines," in *Proc. IEEE Workshop Autom. Speech Recognit. Understand. (ASRU)*, Dec. 2015, pp. 504–511.

[65] X. Ji, M. Yu, J. Chen, J. Zheng, D. Su, and D. Yu, "Integration of multi-look beamformers for multi-channel keyword spotting," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2020, pp. 7464–7468.

[66] X. Ji et al., "An end-to-end far-field keyword spotting system with neural beamforming," in *Proc. IEEE Autom. Speech Recognit. Understand. Workshop (ASRU)*, Dec. 2021, pp. 892–899.

[67] Y. Chen, Q. He, Y. Li, Z. Liu, M. Yang, and J. Huang, "Deep enhancement spotting network for low-complexity keyword spotting in noisy environments," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Apr. 2025, pp. 1–5.

[68] W. Dai, Y. Jiang, Y. Liu, J. Chen, X. Sun, and J. Tao, "CAB-KWS: Contrastive augmentation: An unsupervised learning approach for keyword spotting in speech technology," in *Proc. Int. Conf. Pattern Recognit. (ICPR)*, Dec. 2024, pp. 98–112.

[69] J. Lim and Y. Baek, "Joint framework of curriculum learning and knowledge distillation for noise-robust and small-footprint keyword spotting," *IEEE Access*, vol. 11, pp. 100540–100553, 2023.

[70] J. Mørk, H. Severin Bovbjerg, G. Kiss, and Z.-H. Tan, "Noise-robust keyword spotting through self-supervised pretraining," 2024, *arXiv:2403.18560*.

[71] M. Yu, X. Ji, B. Wu, D. Su, and D. Yu, "End-to-end multi-look keyword spotting," in *Proc. Interspeech*, Oct. 2020, pp. 66–70.

[72] P. M. Reuter, C. Rollwage, and B. T. Meyer, "Multilingual query-by-example keyword spotting with metric learning and phoneme-to-embedding mapping," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Jun. 2023, pp. 1–5.

[73] T.-T. Do, T. Tran, I. Reid, V. Kumar, T. Hoang, and G. Carneiro, "A theoretically sound upper bound on the triplet loss for improving the efficiency of deep distance metric learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 10396–10405.

[74] N. Alon, D. Avdiukhin, D. Elboim, O. Fischer, and G. Yaroslavtsev, "Optimal sample complexity of contrastive learning," in *Proc. 12th Int. Conf. Learn. Represent.*, 2023, pp. 35700–35724.

[75] D. Kim, G. Kim, B. Lee, and H. Ko, "Prototypical knowledge distillation for noise robust keyword spotting," *IEEE Signal Process. Lett.*, vol. 29, pp. 2298–2302, 2022.

[76] Y. Chen, T. Ko, L. Shang, X. Chen, X. Jiang, and Q. Li, "An investigation of few-shot learning in spoken term classification," in *Proc. Interspeech*, Oct. 2020, pp. 2582–2586.

[77] W.-T. Kao, Y.-K. Wu, C.-P. Chen, Z.-S. Chen, Y.-P. Tsai, and H.-Y. Lee, "On the efficiency of integrating self-supervised learning and meta-learning for user-defined few-shot keyword spotting," in *Proc. IEEE Spoken Lang. Technol. Workshop (SLT)*, Jan. 2023, pp. 414–421.

[78] S. G. Patil, P. Jain, P. Dutta, I. Stoica, and J. E. Gonzalez, "POET: Training neural networks on tiny devices with integrated rematerialization and paging," in *Proc. 39th Int. Conf. Mach. Learn.*, 2022, pp. 17573–17583.

[79] O. Khan, G. Park, and E. Seo, "DaCapo: An on-device learning scheme for memory-constrained embedded systems," *ACM Trans. Embedded Comput. Syst.*, vol. 22, no. 5s, pp. 1–23, Oct. 2023.

[80] L. Wulfert et al., "AIfES: A next-generation edge AI framework," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 6, pp. 1–16, Jun. 2024.

[81] M. Rusci and T. Tuytelaars, "On-device customization of tiny deep learning models for keyword spotting with few examples," *IEEE Micro*, vol. 43, no. 6, pp. 50–57, Nov. 2023.

[82] M. Rusci, F. Paci, M. Fariselli, E. Flamand, and T. Tuytelaars, "Self-learning for personalized keyword spotting on ultra-low-power audio sensors," 2024, *arXiv:2408.12481*.

[83] U. Michieli, P. Peso Parada, and M. Ozay, "Online continual learning in keyword spotting for low-resource devices via pooling high-order temporal statistics," in *Proc. Interspeech*, Aug. 2023, pp. 1628–1632.

[84] C. Cioflan, L. Cavigelli, and L. Benini, "Boosting keyword spotting through on-device learnable user speech characteristics," 2024, *arXiv:2403.07802*.

[85] D. Pau, A. Khiari, and D. Denaro, "Online learning on tiny microcontrollers for anomaly detection in water distribution systems," in *Proc. IEEE 11th Int. Conf. Consum. Electron. (ICCE-Berlin)*, Nov. 2021, pp. 1–6.

[86] K. Kopparapu, E. W. Lin, J. G. Breslin, and B. Sudharsan, "TinyFedTL: Federated transfer learning on ubiquitous tiny IoT devices," in *Proc. IEEE Int. Conf. Pervasive Comput. Commun. Workshops other Affiliated Events (PerCom Workshops)*, Mar. 2022, pp. 79–81.

[87] A. Avi, A. Albanese, and D. Brunelli, "Incremental online learning algorithms comparison for gesture and visual smart sensors," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2022, pp. 1–8.

[88] N. Llisterri Giménez, M. Monfort Grau, R. Pueyo Centelles, and F. Freitag, "On-device training of machine learning models on microcontrollers with federated learning," *Electronics*, vol. 11, no. 4, p. 573, Feb. 2022.

[89] M. Ficco, A. Guerriero, E. Milite, F. Palmieri, R. Pietrantuono, and S. Russo, "Federated learning for IoT devices: Enhancing TinyML with on-board training," *Inf. Fusion*, vol. 104, Apr. 2024, Art. no. 102189.

[90] P. Kang, J. Moosmann, S. Bian, and M. Magno, "On-device training empowered transfer learning for human activity recognition," 2024, *arXiv:2407.03644*.

[91] S. S. Saha, S. S. Sandha, and M. Srivastava, "Machine learning for microcontroller-class hardware: A review," *IEEE Sensors J.*, vol. 22, no. 22, pp. 21362–21390, Nov. 2022.

[92] S. Zhu, T. Voigt, F. Rahimian, and J. Ko, "On-device training: A first overview on existing systems," *ACM Trans. Sensor Netw.*, vol. 20, no. 6, pp. 1–39, Oct. 2024.

[93] D. Rossi et al., "Vega: A ten-core SoC for IoT endnodes with DNN acceleration and cognitive wake-up from MRAM-based state-retentive sleep mode," *IEEE J. Solid-State Circuits*, vol. 57, no. 1, pp. 127–139, Jan. 2022.

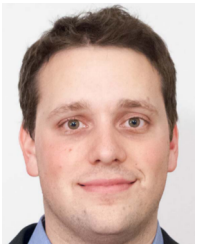
[94] M. Spallanzani, G. Rutishauser, M. Scherer, A. Burrello, F. Conti, and L. Benini. (2022). *Quantlab: A Modular Framework for Training and Deploying Mixed-Precision NNS*. [Online]. Available: https://cms.tinyml.org/wp-content/uploads/talks2022/Spallanzani-Matteo-Hardware.pdf
[95] P. Bartoli, T. Bondini, C. Veronesi, A. Giudici, N. Antonello, and F. Zappa, "End-to-end efficiency in keyword spotting: A system-level approach for embedded microcontrollers," 2025, *arXiv:2509.07051*.
[96] P. Bartoli, C. Veronesi, A. Giudici, D. Siorpaes, D. Trojaniello, and F. Zappa, "Benchmarking energy and latency in TinyML: A novel method for resource-constrained AI," 2025, *arXiv:2505.15622*.
[97] J. Bokstaller, J. Schneider, and J. V. Brocke, "Estimating SoC, SoH, or RuL of rechargeable batteries via IoT: A review," *IEEE Internet Things J.*, vol. 11, no. 5, pp. 7559–7582, Mar. 2024.
[98] F. Paissan et al., "Structured sparse back-propagation for lightweight on-device continual learning on microcontroller units," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2024, pp. 2172–2181.
[99] D. Nadalini, M. Rusci, L. Benini, and F. Conti, "Reduced precision floating-point optimization for deep neural network on-device learning on microcontrollers," *Future Gener. Comput. Syst.*, vol. 149, pp. 212–226, Dec. 2023.
[100] P. H. C. Pereira, W. Beccaro, and M. A. Ramírez, "Evaluating robustness to noise and compression of deep neural networks for keyword spotting," *IEEE Access*, vol. 11, pp. 53224–53236, 2023.
[101] G. Lee, S. Lee, and H. Kim, "ACC: Adaptive compression framework for efficient on-device CNN training," in *Proc. IEEE 6th Int. Conf. AI Circuits Syst. (AICAS)*, Apr. 2024, pp. 472–476.
[102] P. Luo, J. Hou, M. Yuan, G. Wu, Y. Yao, and X. Li, "F2Zip: Finetuning-free model compression for scenario-adaptive embedded vision," in *Proc. 22nd ACM Conf. Embedded Networked Sensor Syst.*, 2024, pp. 15–27.
[103] X. Zhu, J. Wang, K. Sato, and Z. Zheng, "Adaptive model compression for efficient federated learning in IoT systems," *IEEE Internet Things J.*, vol. 12, no. 14, pp. 26155–26168, Jul. 2025.
[104] Y. Liu et al., "Deep anomaly detection for time-series data in industrial IoT: A communication-efficient on-device federated learning approach," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6348–6358, Apr. 2021.
[105] F. Wu et al., "Participant and sample selection for efficient online federated learning in UAV swarms," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 21202–21214, Jun. 2024.
[106] M. M. Grau, R. P. Centelles, and F. Freitag, "On-device training of machine learning models on microcontrollers with a look at federated learning," in *Proc. Conf. Inf. Technol. Social Good*, Sep. 2021, pp. 198–203.
[107] Y. Gao et al., "End-to-end evaluation of federated learning and split learning for Internet of Things," in *Proc. Int. Symp. Reliable Distrib. Syst. (SRDS)*, Sep. 2020, pp. 91–100.
[108] V. Ninkovic, D. Vukobratovic, D. Miskovic, and M. Zennaro, "COMSPLIT: A communication-aware split learning design for heterogeneous IoT platforms," *IEEE Internet Things J.*, vol. 12, no. 5, pp. 5305–5319, Mar. 2025.
[109] X. Feng, R. Jia, C. Luo, V. C. M. Leung, and W. Xu, "SLwF: A split learning without forgetting framework for Internet of Things," *IEEE Internet Things J.*, vol. 12, no. 9, pp. 12008–12020, May 2025.



Cristian Cioflan (Graduate Student Member, IEEE) received the B.Eng. degree in electrical engineering and information technology from the University Politehnica of Bucharest, Romania, in 2018, and the M.Sc. degree from the Swiss Federal Institute of Technology Zürich (ETH Zürich), Switzerland, in 2020, where he is currently pursuing the Ph.D. degree with the Digital Circuits and Systems Group of Prof. Luca Benini.

His research interests include on-device continual learning, audio processing in low-power embedded systems, and neural architecture search for energy-efficient inference.

Mr. Cioflan has received the Best Paper Award at the TinyML Research Symposium 2024.



Lukas Cavigelli (Member, IEEE) received the B.Sc., M.Sc., and Ph.D. degrees in electrical engineering and information technology from ETH Zürich, Switzerland, in 2012, 2014, and 2019, respectively.

After spending a year as a Post-Doctoral Researcher with ETH Zürich, he joined Huawei's Zurich Research Center in 2020. His research interests include deep learning, computer vision, embedded systems, and low-power integrated circuit design.

Dr. Cavigelli has received the Best Paper Award at the VLSI-SoC and the ICDSC Conferences in 2013 and 2017, the Best Student Paper Award at the Security+Defense Conference in 2016, and the Donald O. Pederson Best Paper Award in 2019.



Manuele Rusci received the Ph.D. degree in electronic engineering from the University of Bologna, Italy, in 2018.

He is currently a tenure-track Assistant Professor with KU Leuven, Leuven, Belgium. He has been a Post-Doctoral Fellow and a Marie Skłodowska-Curie Actions Post-Doctoral Grantee with KU Leuven, Leuven, Belgium. His main research interests include low-power, AI-powered smart sensors and on-device continual learning.



Miguel de Prado received the joint M.Sc. degree (First Class Distinction) from the University of Southampton, Southampton, U.K., and TU Kaiserslautern, Kaiserslautern, Germany, and the Ph.D. degree from ETH Zürich, Switzerland, under Prof. Luca Benini, focusing on optimizing neural network deployment and domain adaptation on edge devices.

He is a Senior Research Engineer with VERSES, Los Angeles, CA, USA, and oversees the successful development of Research and Development European Projects. His work focuses on revolutionizing edge AI with cutting-edge hardware and software, pushing the boundaries of technology, and making AI accessible and transformative for society.



Luca Benini (Fellow, IEEE) is currently the Chair of Digital Circuits and Systems with ETH Zürich, Zürich, Switzerland, and a Full Professor with the University of Bologna, Bologna, Italy. He is also active in the area of energy-efficient smart sensors and sensor networks. His research interests are in energy-efficient systems and multi-core SoC design.

Mr. Benini is a fellow of ACM and a member of the Academia Europaea.